



BluePrint VS Code Extension
SE-801: Software Project Lab III
Technical Report

Submitted by

Sejuti Sharmin Siddiqui
Roll: BSSE 1420

Supervised by

Dr. Kazi Muheymin-Us-Sakib
Professor
Institute of Information Technology
University of Dhaka

Submission Date: 10-10-2026

Letter of Transmittal

October 10, 2026
BSSE 4th Year Exam Committee
Institute of Information Technology
University of Dhaka

Subject: Final Report Submission of BluePrint VS Code Extension

Dear Sir,

I am submitting the technical report for my Software Project Lab 3, BluePrint: Human in the Loop Persistent Architectural Knowledge for AI-Assisted Development. This report comprehensively covers the project's technical details, including design, implementation, and testing phases. While I've tried to do my best, I welcome your feedback for improvement.

Thank you for your consideration.

Sincerely,
Sejuti Sharmin Siddiqui
BSSE 1420
Institute of Information Technology
University of Dhaka

Supervisor's Signature

Acknowledgement

I would like to express my sincere gratitude to the Institute of Information Technology, University of Dhaka, for the opportunity to work on the project. This journey has been both rewarding and enlightening. A special thanks to my supervisor, Dr. Kazi Muheymin-Us-Sakib, Professor at the Institute of Information Technology, whose guidance and encouragement were invaluable throughout the project. His insights helped shape the project's direction and fueled my commitment to excellence. I also appreciate my classmates' camaraderie and valuable feedback, which added depth to this experience and helped refine the project.

Abstract

AI coding assistants such as GitHub Copilot, Cursor and Claude Code increasingly write production code, but they see only the prompt in front of them, not the architecture of the system. When an assistant makes an unrequested architectural choice, such as adding a database, service or dependency, the choice is easily missed in review and its rationale is never recorded. Over time this causes architectural drift, loss of design intent between sessions, and erosion of developer control. Existing instruction files are static, and decision-record tools do not verify generated code against stored decisions. This report presents BluePrint, a Visual Studio Code extension that keeps architecture and decisions alongside the code and checks both prompts and code changes against them, leaving every decision to a person. BluePrint follows a Constraints → Conformance → Knowledge cycle: a setup wizard produces an architecture document (ARCH.md) and Architectural Decision Records (ADRs); a pre-check compares prompts with recorded decisions; and a two-pass compliance review summarises uncommitted changes with Tree-sitter, retrieves relevant ADRs through hybrid embedding and lexical ranking, and uses a large language model (LLM) to report violations and unplanned components. Developer resolutions become new or updated ADRs, with version history, role-based approval and an audit trail. All 169 automated tests pass, live LLM tests met their criterion (4/4 runs), and manual acceptance testing found three defects, all fixed. Limitations include reliance on LLM judgement, the need for a git repository and API key, and incomplete acceptance testing.

Table of Contents

1. Introduction.....	1
1.1 Problem Statement.....	1
1.2 Project Description	1
1.3 Project Scope	2
1.4 Deliverables.....	2
2. Requirements Analysis	3
2.1 Normal Requirements.....	3
2.2 Expected Requirements	3
2.3 Exciting Requirements	4
2.4 Stakeholders	4
3. Scenario-Based Modeling.....	5
3.1 Usage Scenario	5
3.1.1 Installation.....	5
3.1.2 Project Setup	5
3.1.3 Pre-generation Prompt Check	5
3.1.4 Post-generation Compliance Review	5
3.1.5 Browsing Architectural Knowledge.....	5
4. AI Engineering Design.....	6
4.1 AI Pipeline.....	6
4.2 Model Selection.....	7
4.3 Prompt Design.....	7
5. Component-Level Design	8
5.1 Architecture Overview	8
5.2 Design Classes of the Problem Domain	9
5.3 Design Classes: Responsibilities and Collaborators.....	12
5.4 Persistent Data Sources	16
5.5 Behavioural Representations	16
5.5.1 State Diagrams	16
5.5.2 Sequence Diagram: Reviewing a Code Change.....	18
5.5.3 Event-Driven Collaboration Diagram	19
6. Interface Design	21
6.1 User Interface Objects and Actions	21
6.2 Events and Interface State Changes.....	23
6.3 Interface States as Seen by the User.....	26
6.3.1 Status Bar and Sidebar	26
6.3.2 Hub.....	27
6.3.3 Setup Wizard.....	28
6.3.4 Compliance Review	30
6.3.5 Pre-Check and Add Decision	34
6.3.6 ARCH.md Viewer, Audit Trail and Team & Roles	35

7. Testing	39
7.1 Testing Approach and Strategy	39
7.2 Item Pass/Fail Criteria	39
7.3 Risks and Contingencies	39
7.4 Test Cases	40
8. User Manual	44
8.1 System Requirements	44
8.2 Installation	44
8.2.1 Installing a package	44
8.2.2 Building from source	45
8.2.3 Checking the installation	45
8.3 Obtaining an API Key	45
8.4 Setting Up a Project	45
8.5 Using BluePrint	46
8.5.1 Checking a prompt before coding	46
8.5.2 Reviewing code changes	47
8.5.3 Recording a decision	48
8.5.4 Browsing decisions and the audit trail	48
8.5.5 Viewing and maintaining ARCH.md	48
8.6 Working in a Team	48
8.7 Command Reference	49
8.8 Settings	49
8.9 Troubleshooting	50
8.10 Uninstalling	50
9. Conclusion	51
9.1 Limitations	51
9.2 Future Work	51
References	52

Table of Figures

Figure 4.1: AI Pipeline.....	6
Figure 5.1: Layered architecture of BluePrint	9
Figure 5.2: Domain classes for architectural knowledge	11
Figure 5.3: Domain classes for a compliance review	12
Figure 5.4: State diagrams of (a) an ADR and (b) an architecture component.....	17
Figure 5.5: State diagram of a compliance review (status bar).....	18
Figure 5.6: Sequence diagram for reviewing a code change	19
Figure 5.7: Event-driven collaboration diagram	20
Figure 6.1: Interface objects and the navigation between them.....	23
Figure 6.2: State diagram of the Setup Wizard	25
Figure 6.3: State diagrams of (a) the Compliance Review panel and (b) the Pre-Check panel	26
Figure 6.4: The status bar item in each state (reconstruction)	27
Figure 6.5: The BluePrint sidebar: ADR Browser and Audit Trail (reconstruction)	27
Figure 6.6: The Hub.....	28
Figure 6.7: Setup Wizard, first steps.....	29
Figure 6.8: Setup Wizard, constraint questions	30
Figure 6.9: Compliance Review results: violation and planned component.....	31
Figure 6.10: Compliance Review results (continued): new component and checked files.....	32
Figure 6.11: Compliance Review outcomes without items	33
Figure 6.12: Pre-Check result for a conflicting prompt	34
Figure 6.13: Add Decision with a suggested replacement.....	35
Figure 6.14: ARCH.md Viewer with recent changes highlighted	36
Figure 6.15: Audit Trail panel, Activity tab.....	37
Figure 6.16: Team & Roles panel	38

Table of Tables

Table 5.1: Problem-domain classes	10
Table 5.2: CRC cards, UI layer	13
Table 5.3: CRC cards, coordination layer	14
Table 5.4: CRC cards, agent layer	15
Table 5.5: CRC cards, infrastructure layer	16
Table 6.1: User interface objects and their actions	22
Table 6.2: Events and the interface state changes they cause	24
Table 7.1: Testing risks and contingencies	40
Table 7.2: Test cases with outcomes.....	43
Table 8.1: Installation packages.....	44
Table 8.2: Supported providers.....	45
Table 8.3: Files created by Blueprint in the project.....	46
Table 8.4: Compliance review results and how to resolve them.....	47
Table 8.5: Blueprint commands (command palette).....	49
Table 8.6: Blueprint settings	50
Table 8.7: Common problems and solutions	50

BluePrint: Human in the Loop Persistent Architectural Knowledge for AI-Assisted Development

1. Introduction

The rise of AI coding agents has changed how software gets built. Tools like GitHub Copilot, Cursor, and Claude Code have made it possible to generate working code from a single sentence and developers increasingly depend on them for everything from small edits to entire features. This shift has made development faster, but it has also introduced a new kind of risk: these agents are skilled at producing code that fits its immediate surroundings, yet they have no lasting sense of the system's overall design. Decisions that used to be made deliberately, such as which storage layer to use, how services should be separated, and what conventions the team follows are now often made implicitly, with no record of why.

BluePrint is proposed to close this gap. It sits alongside a developer's existing AI-assisted workflow inside VSCode, giving projects a persistent, evolving record of their architecture and the reasoning behind it, so that speed from AI-generated code does not come at the cost of architectural clarity and ownership.

1.1 Problem Statement

AI coding agents such as GitHub Copilot, Cursor, and Claude Code have become deeply embedded in everyday software development. These tools generate locally coherent, context-aware code, but they have no persistent memory of the architectural reasoning behind a project. Each generation is, in effect, architecturally stateless: the agent does not know why a previous decision was made, and it does not record why the current one is being made.

This creates a quiet but compounding problem. A developer may ask for what feels like a small change "add caching to this endpoint" and the agent may respond by introducing a new storage layer, a new dependency, or a new service boundary that was never discussed. Individually, each of these choices seems reasonable. Collectively, over dozens of prompts and weeks of development, they add up to architectural drift: a system whose actual structure has quietly diverged from what the team believes it to be.

Two further effects compound this drift:

- **Context degradation** — because AI agents do not retain design intent across sessions, the same architectural questions get re-litigated, sometimes inconsistently, as a project grows.
- **Loss of developer control** — architectural decisions that were traditionally made deliberately, with rationale documented for future maintainers, are instead made implicitly by an agent and left unrecorded.

The motivation behind BluePrint is that developers should not have to choose between the productivity of AI-assisted coding and ownership of their system's architecture. BluePrint is motivated by the belief that architectural knowledge, the why behind a system's structure is as important an artifact as the code itself and that it should be captured continuously and cheaply as a natural byproduct of the development workflow rather than a separate documentation chore.

1.2 Project Description

BluePrint is a VSCode extension that makes AI-assisted software development architecture-aware by maintaining a persistent, continuously updated record of a project's architecture and the reasoning behind it.

The system takes as input:

- A natural-language system description provided by the developer at project setup
- Developer prompts submitted to an AI coding assistant
- The code changes generated as a result
- Architectural decisions and reasoning the developer provides when prompted

From these inputs, BluePrint produces and maintains two living artifacts:

- **ARCH.md** — a structured, human-readable architectural blueprint of the project (system overview, components, data flow, constraints, and open questions)
- **A knowledge base of Architectural Decision Records (ADRs)** — individually stored, timestamped decisions in MADR format, each with its own context, decision, and consequences

Internally, BluePrint is implemented as a multi-agent system coordinated by a central orchestrator, comprising an Architecture Agent, a Constraint Elicitation Agent, a Compliance Agent, and a Retrieval Agent, with the VSCode extension itself acting purely as a UI shell. The workflow follows a Constraints → Conformance → Knowledge cycle. At first, the constraints are elicited up front, then the generated code is checked for conformance to those constraints, and lastly, every decision, whether a conformance approval, a violation resolution, or an architectural extension, is captured as reusable knowledge for future development cycles.

Critically, BluePrint is designed to be AI-tool agnostic. It does not integrate with the internal API of any specific coding assistant. Instead, it observes code changes via file diffs after the developer explicitly requests a review, allowing it to work alongside Copilot, Cursor, Claude Code, or manually written code alike.

1.3 Project Scope

- A VSCode extension providing setup, review, and browsing interfaces for architectural knowledge
- Generation of an initial ARCH.md from a natural-language system description
- A structured constraint-elicitation flow that produces ADRs
- Post-generation compliance checking of code diffs against recorded constraints, in two passes (violation detection, then extension detection)
- Optional pre-generation prompt-level conflict checking against existing ADRs
- Local, project-embedded storage of all architectural artifacts (ARCH.md, ADRs, structured indices, audit history) — version-controlled and portable with the repository
- Retrieval-augmented context assembly, using local embeddings to surface relevant past ADRs
- Operation entirely on free-tier LLM APIs (e.g., Google Gemini free tier), with no dependency on paid inference

1.4 Deliverables

The primary deliverable of this project is a VSCode extension that maintains a persistent, continuously updated architectural knowledge base alongside AI-assisted development. This includes the extension's setup, review, and browsing interfaces; the underlying multi-agent system and the two artifacts it produces and maintains for each project which are a structured, human-readable ARCH.md and a locally stored knowledge base of Architectural Decision Records (ADRs) in MADR format.

2. Requirements Analysis

Quality Function Deployment (QFD) is a technique used to translate user needs into prioritized product requirements. Here, it is used to classify BluePrint's requirements into Normal, Expected, and Exciting categories based on how each affects developer satisfaction.

2.1 Normal Requirements

- **Architecture generation:** Generate an initial ARCH.md from a developer-provided natural-language system description.
- **Constraint elicitation:** Conduct a structured constraint-elicitation dialogue and store each surfaced constraint as an ADR, subject to developer approval.
- **Violation detection:** Detect architectural violations in generated code by comparing a diff against stored constraints and ADRs.
- **Violation resolution:** When a violation is detected, present the developer with the choice to update the architecture or modify the code, and record the developer's reasoning as a new ADR.
- **Extension detection:** Detect architectural extensions (new, non-contradicting structural elements) in generated code that has passed violation checking, and prompt the developer to confirm and record reasoning.
- **Contextual retrieval:** Retrieve relevant past ADRs as context for compliance checks, using both semantic and lexical similarity over ADR embeddings.
- **Local storage:** Store all architectural artifacts locally within the project repository in version-controllable formats.
- **Developer interface:** Provide a VSCode UI for setup, reviewing compliance results, and browsing ADRs.
- **Version history:** Maintain a local version history of ARCH.md, allowing changes to be reviewed or reverted.
- **Role-based access control:** Resolve the developer's identity from their local Git configuration and role from a version-controlled Roles Manifest, routing architecture-changing decisions (new ADRs, "Update Architecture" resolutions, confirmed extensions) to an Architect for approval before they are finalized.

2.2 Expected Requirements

- **Free-tier operation:** Operate entirely on free-tier LLM APIs, without requiring the developer to hold a paid subscription.
- **Non-destructive updates:** Preserve manually edited sections of ARCH.md when applying agent-generated updates, using targeted patch edits rather than full regeneration.
- **Developer confirmation:** Never silently modify code or architecture files without developer confirmation.
- **Responsiveness:** Remain responsive and non-blocking within the VSCode editor during agent operations.
- **Tool independence:** Work independently of which AI coding assistant (or none) the developer used to generate the code being reviewed.
- **Data locality:** Keep all developer data (code, prompts, architectural knowledge) local to the project rather than transmitting it to a third-party service beyond the LLM API call itself.

2.3 Exciting Requirements

- **Pre-generation conflict check:** Provide a prompt-level conflict check — using semantic search over embedded ADRs — that warns the developer of likely architectural conflicts before code is even generated, and lets the developer attach the retrieved ADRs to the prompt as additional context for the AI assistant.
- **Decision audit trail:** Maintain a searchable timeline view of every architectural decision made over the project's lifetime, with links back to the ADRs and code changes that prompted them.
- **Hybrid retrieval:** Apply keyword-based filtering on component names alongside semantic retrieval, so structurally relevant ADRs are not missed even when they are not semantically similar to the current diff.
- **Diff summarization:** Summarize and reduce raw code diffs before sending them to an LLM, to keep requests within free-tier context and rate limits.

2.4 Stakeholders

Stakeholders are individuals or groups who are actively involved in the project, process, or system and directly interact with it. Their roles and responsibilities are clearly defined, and they are impacted by the system's operations, changes, and performance. The primary stakeholder for BluePrint are software developers, who might develop any project using an AI assisted workflow.

3. Scenario-Based Modeling

Scenario-based modeling is a method used in software development to describe how a system behaves in response to real-world interactions, events, and conditions. By walking through concrete usage scenarios and mapping out how actors engage with the system, this approach helps surface functional requirements early, clarify system boundaries, and validate that the design will actually meet developer needs before implementation begins.

3.1 Usage Scenario

BluePrint is used alongside a developer's normal AI-assisted coding workflow, inserting architecture-awareness at the points where architectural decisions are most likely to be made. The typical usage scenario is as follows:

3.1.1 Installation

Developers install BluePrint from the VSCode Marketplace like any other extension. No separate service or account is required. All architectural data is stored inside the project repository.

3.1.2 Project Setup

On first use in a project, the developer provides a natural-language description of the system through the Setup Wizard. BluePrint's Architecture Agent uses this description to generate an initial ARCH.md, and the Constraint Elicitation Agent then runs a structured, branching conversation to surface architectural constraints — technology choices, scalability expectations, team conventions, integration requirements, and non-functional requirements each of which the developer reviews and approves as an ADR.

3.1.3 Pre-generation Prompt Check

Before submitting a prompt to an AI coding assistant (e.g., Copilot, Cursor, Claude Code), the developer can optionally run a pre-check from the extension UI. BluePrint computes a local embedding of the prompt and runs a semantic search over the embedded ADR knowledge base to retrieve the top-K most relevant past decisions, then compares the prompt's intent against those ADRs and against ARCH.md constraints, warning the developer if it is likely to conflict with a past architectural decision — before any code is generated. The developer can revise the prompt in response to a warning, or proceed as-is. Either way, the developer can choose to attach the retrieved, relevant ADRs to the prompt as additional context for the AI assistant, increasing the odds that the generated code aligns with decisions the project has already made.

3.1.4 Post-generation Compliance Review

After the AI assistant generates code, the developer clicks a “Review this change architecturally” button (or BluePrint auto-triggers on new file creation). BluePrint computes and summarizes the diff, retrieves relevant past ADRs, and runs its two-pass Compliance Agent. If a violation is found, the developer chooses to update the architecture or modify the code and records their reasoning. If no violation is found but the change introduces a new structural element, the developer is asked to confirm the extension; both outcomes are saved as ADRs and reflected in ARCH.md.

3.1.5 Browsing Architectural Knowledge

At any point, the developer can open the ARCH.md Viewer to see the current architecture with recently touched sections highlighted, browse and search all ADRs in the ADR Browser, or review the Decision Audit Trail to see the full history of architectural decisions made over the project's lifetime, along with the reasoning behind each one.

4. AI Engineering Design

BluePrint's intelligence layer is implemented as a multi-agent system: a set of specialist agents, each responsible for one narrow judgment task, coordinated by a single orchestrator that invokes them in response to developer actions inside the editor. This section covers the pipeline architecture connecting these agents and the models selected for them.

4.1 AI Pipeline

- **Architect Agent:** Synthesizes an initial system blueprint from a free-text description.
- **Constraint Elicitor Agent:** Converts interview answers into candidate architectural decision records.
- **Retriever Agent:** Ranks and narrows architectural decision records to the most relevant subset. This is the one agent in the system that does not call the external LLM API
- **Compliance Auditor Agent:** Detects whether a code change violates an established decision.
- **Extension Detector Agent:** Detects whether a code change introduces a new structural component.
- **Pre-Check Sentinel Agent:** Checks a developer's stated intent against architecture before code is written.

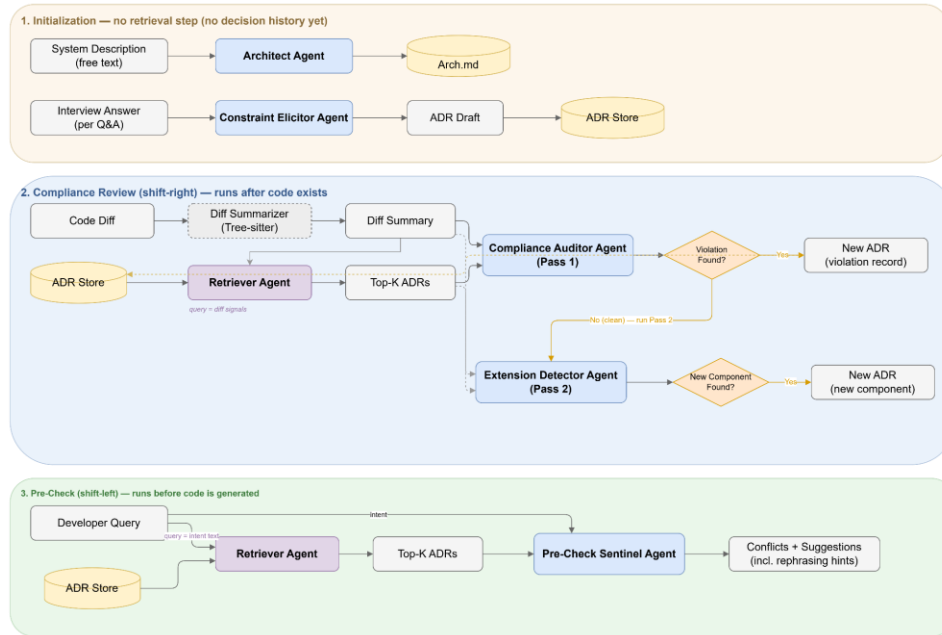


Figure 4.1: AI Pipeline

The agents work together to establish three system workflows. First, during initialization, the Architect Agent produces the first blueprint, and the Constraint Elicitor Agent drafts a decision record per interview answer. This is the only workflow with no retrieval step, since no decision history yet exists. Second, during compliance review, run after code exists, the Compliance Auditor Agent and Extension Detector Agent are invoked as two separate calls rather than one, since combining "was a rule violated" with "is this a new subsystem" in a single response was found to confuse the resolution workflow. Third, during pre-check, run before code is generated, the Pre-Check Sentinel Agent screens the developer's stated intent against retrieved decisions, making this a preventive check rather than a detective one.

4.2 Model Selection

Every specialist agent depends on a single abstract interface rather than a vendor-specific SDK, making the underlying language model a user-facing configuration choice rather than a code dependency. Default models such as **Gemini 2.5 Flash**, **gpt 4o (Open AI)** or **Groq API** can be selected for low cost and latency rather than maximum capability. Reflecting the nature of the task: every agent call is a short, structured extraction or classification, not open-ended generation, so small-to-mid tier models are sufficient.

On the other hand, the Retriever Agent's semantic signal comes from a small, local sentence-embedding model, **all-MiniLM-L6-v2**, run in-process via ONNX rather than a hosted embedding API. A hosted-embedding design would require a second, provider-specific integration layered on top of the existing abstraction. A single local model keeps retrieval fully offline, provider-independent, and free of per-query cost, at the price of a documented fallback — retrieval degrades to lexical-only scoring if the local model fails to load.

All six agents in the current design share one provider and model chosen once at setup, rather than a tiered assignment of cheaper models to low-stakes extraction and stronger models to judgment-heavy checks; and no sampling parameters, such as temperature or top-p, are explicitly tuned, relying instead on each provider's default behavior.

4.3 Prompt Design

Each specialist agent's prompt is built from a shared set of design principles, applied consistently across the system.

- Role framing. Every agent is given a narrow professional persona — for example, the Compliance Auditor Agent is framed as "an architectural compliance reviewer" — rather than a generic assistant framing, priming the model toward the specific judgment required.
- Schema-as-example generation. Rather than few-shot examples, each prompt states its required JSON schema inline, with descriptive placeholder text per field, teaching the model both the expected shape and level of detail in a single instruction.
- Explicit inclusion and exclusion rules. Judgment-heavy agents are given explicit do/do-not rules rather than being left to infer boundaries, which was the single largest lever against false positives observed during testing.
- Concrete severity rubrics. Where an agent must grade severity, the meaning of each grade is defined in concrete terms, keeping results comparable across separate calls and across providers.
- Asymmetric tie-breakers. Where false positives and false negatives carry different costs, the prompt states an explicit bias in the direction that matters. The Extension Detector Agent is told to lean toward reporting, since a missed extension lets architecture silently drift, while the Compliance Auditor Agent is told to stay silent on anything not clearly covered, since a false violation actively blocks the developer.
- Sectioned, labeled context. All injected context is assembled into clearly headed blocks ending in one explicit question, rather than left as freeform text, keeping prompts easy to extend.
- Context budgeting. Every injected list is capped, including the top five decision records, bounded diff summaries, and character limits on raw diff text, to control cost and keep the agent's attention on the highest-signal material.
- Defensive parsing as part of the contract. Because no prompt can guarantee compliance, every agent's prompt is paired with fence-stripped JSON parsing and a named safe default, which is a prompt design decision as much as an implementation one.

5. Component-Level Design

This section refines the architecture of BluePrint into its components. BluePrint is a Visual Studio Code extension written in TypeScript; it keeps a project's architecture (ARCH.md) and its Architectural Decision Records (ADRs) next to the code and uses a large language model (LLM) to check prompts and code changes against them. Section 5.1 gives a simplified overview, 5.2 identifies the classes that model the problem domain, 5.3 describes every design class with its responsibilities and collaborators, 5.4 lists the persistent data sources, and 5.5 elaborates the behaviour of the main classes.

5.1 Architecture Overview

BluePrint is organised in four layers (Figure 5.1). The **UI layer** registers the extension's commands, sidebar views, status bar item and webview panels. The **coordination layer** turns user actions and workspace events into pipelines: the Orchestrator routes each event to the agents it needs, and the DecisionService is the single entry point through which every new decision is recorded. The **agent layer** holds the LLM-backed agents and the analysis steps that prepare their input. The **infrastructure layer** wraps everything outside the extension: the LLM providers, local embeddings, source parsing, the project files and the VS Code storage.

Calls only go downwards. The coordination layer reports back to the UI through two hooks, a state change (idle, checking, OK, violation) and a data change, so the UI never depends on how a pipeline works.

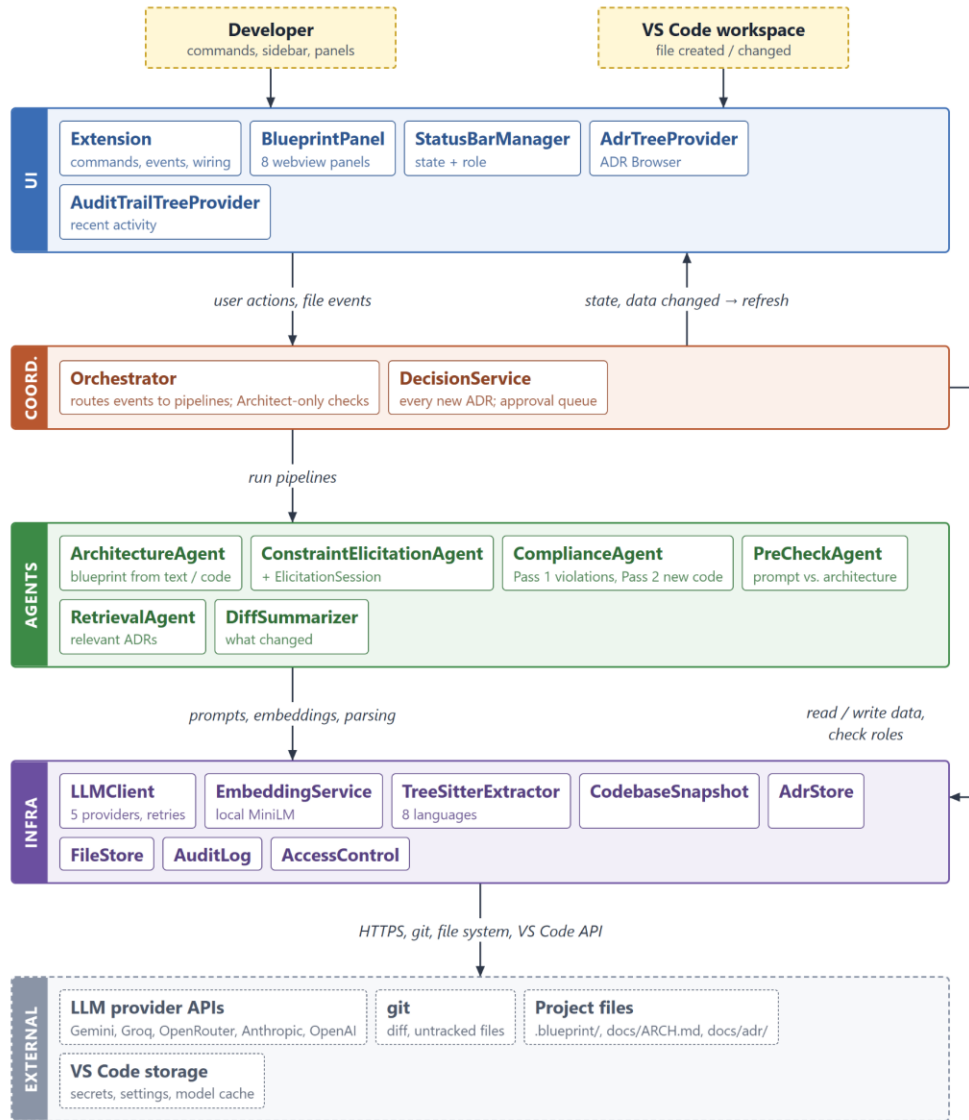


Figure 5.1: Layered architecture of BluePrint

5.2 Design Classes of the Problem Domain

The problem domain is architectural knowledge and how code changes are checked against it. Table 5.1 lists the classes that model it; Figures 5.2 and 5.3 show their attributes and relationships. These are data classes: they carry no behaviour and are shared by all layers.

Class	Represents	Key attributes
ArchBlueprint	The project's architecture, rendered as ARCH.md	system overview, components, data flow, constraints, open questions
ArchComponent	One component of the architecture	name, responsibility, technology, status (planned or implemented), files that implement it

ADR	An Architectural Decision Record	title, context, decision, consequences, status, proposer and reviewer, the effect it has on ARCH.md, the ADR it supersedes
ArchEffect	What accepting an ADR changes in ARCH.md	add a component, add a constraint, replace a constraint, mark a component implemented
RolesManifest	The team's roles	Architect and Developer e-mail addresses
AuditEntry	One event in the decision audit trail	time, event type, summary, actor, changed files, linked ADR or review result
ArchHistoryEntry	An earlier version of ARCH.md	time, reason for the change, sections it touched
DiffSummary	The uncommitted code change under review	added, modified, renamed and deleted files; new imports and declarations; added and removed dependencies; a diff excerpt
ComplianceResult	The outcome of one review	violations (Pass 1); extensions, planned components now in code and checked files (Pass 2); ADRs used as context
ViolationDetail	A change that breaks a constraint or ADR	constraint or ADR id, description, severity, code location
ExtensionDetail	New code that is neither a planned nor an implemented component	name, responsibility, technology, rationale, files
PlannedComponentMatch	New code that is a planned component	component name, files, rationale
CheckedFile	An added file that is neither, with the reason	file, reason

Table 5.1: Problem-domain classes

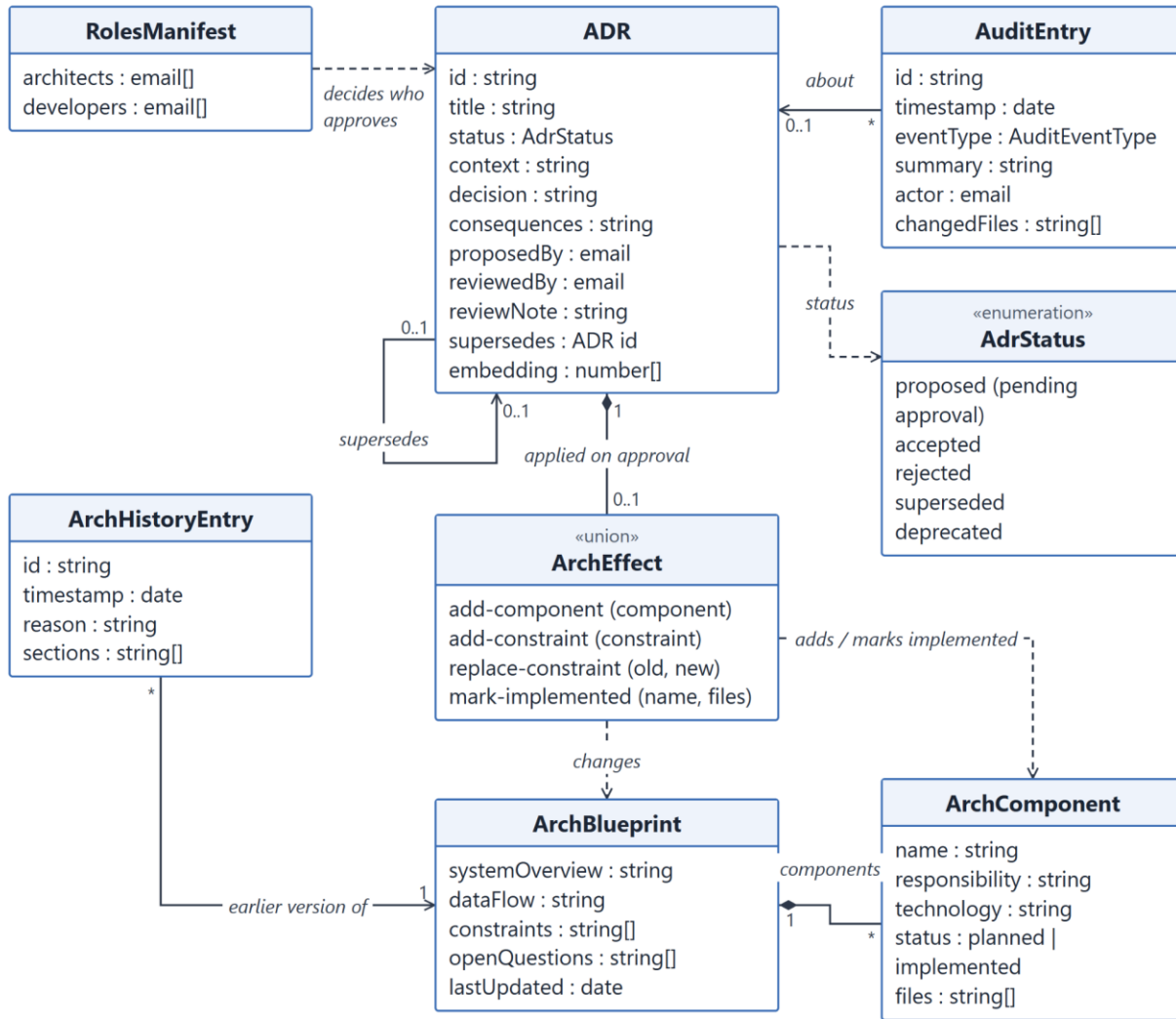


Figure 5.2: Domain classes for architectural knowledge

An ADR changes the architecture only when it is accepted: its ArchEffect is applied to ArchBlueprint at that moment, so a Developer's proposal leaves ARCH.md untouched until an Architect approves it. Each change to ARCH.md is preceded by an ArchHistoryEntry, which makes every change revertible. A component is **planned** when it was described (for example during initialization) but has no code yet, and **implemented** once code for it exists.

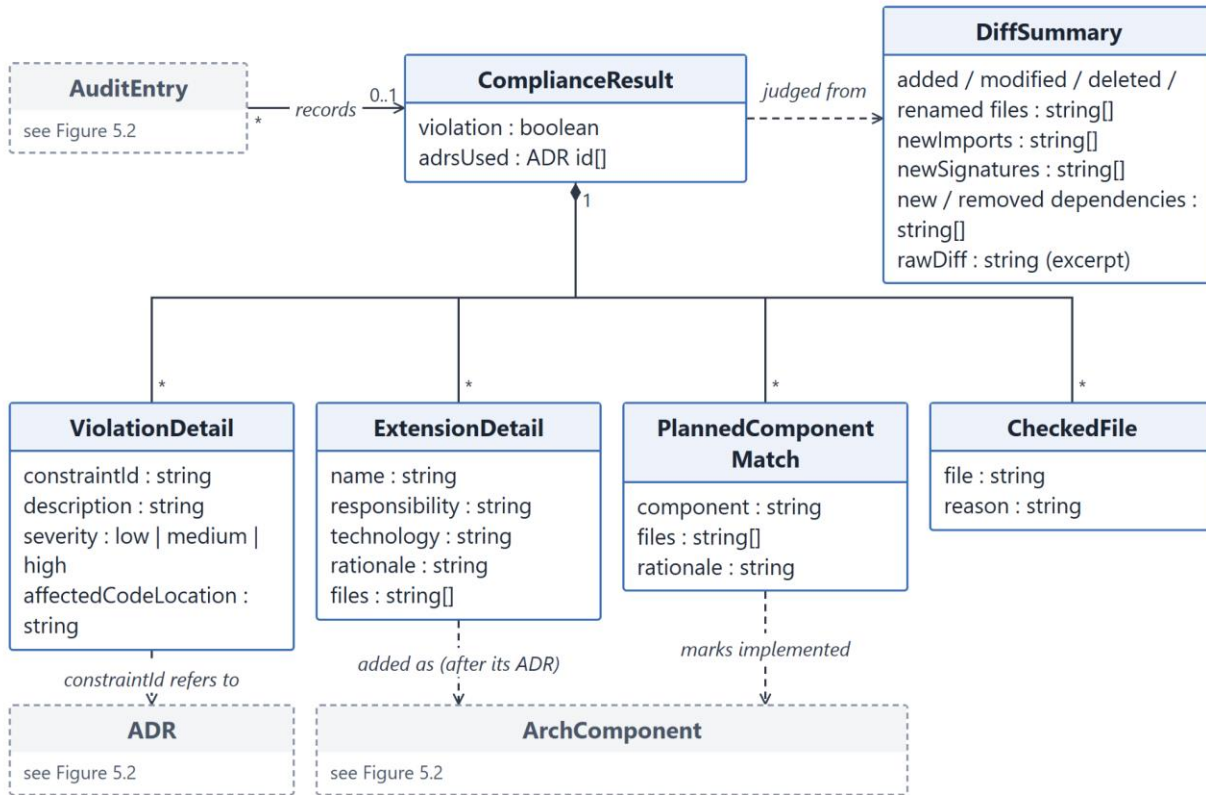


Figure 5.3: Domain classes for a compliance review

A review produces one ComplianceResult from one DiffSummary. Pass 1 contributes the violations; Pass 2 sorts the new code into the three remaining groups. A violation refers to an existing ADR or constraint; an extension becomes a new component once its ADR is accepted; a planned match marks an existing component implemented without an ADR, because that component was already decided.

5.3 Design Classes: Responsibilities and Collaborators

Tables 5.2 to 5.5 give the Class-Responsibility-Collaborator (CRC) description of every design class, layer by layer. Collaborators are the classes a class calls directly; «module» marks a file of functions rather than a class. The HTML pages of the webview panels are covered in Section 6.

Class	Responsibilities	Collaborators
Extension «module»	- Register commands, sidebar views and the status bar item - Open webview panels and relay their messages to the Orchestrator - Run the setup wizard and the LLM provider / API key change, validating each key - Offer a review when new files are created - Watch ARCH.md, the ADR index, roles.json and the audit log, and refresh the UI	Orchestrator, BlueprintPanel, StatusBarManager, ADRTreeProvider, AuditTrailTreeProvider, LLMClient, AccessControl, ADRStore, FileStore, AuditLog

BlueprintPanel	• Keep one webview panel per type (Hub, Setup, Compliance Review, Pre-Check, Add Decision, ARCH.md Viewer, Audit Trail, Team & Roles) • Load each panel's HTML with a content-security nonce • Pass messages between the panel and the extension	Extension
StatusBarManager	• Show the state: not initialized, idle with the pending-approval count, checking, OK, violation • Bind each state's click to the right action (a violation reopens the last review) • Show the user's identity and role in the tooltip	Extension
AdrTreeProvider	• List ADRs in the sidebar grouped by status, pending approvals first • Filter ADRs by text; clear the filter • Offer Approve / Reject to Architects; mark superseded ADRs	Extension
AuditTrailTreeProvider	• List the 20 most recent audit-log entries, newest first • Open the linked ADR, or the full audit trail, when an entry is clicked	Extension

Table 5.2: CRC cards, UI layer

Class	Responsibilities	Collaborators
Orchestrator	• Route events (project initialized, prompt submitted, code generated, review requested) to agent pipelines • Run compliance Pass 1 and Pass 2 in parallel • Resolve violations, register extensions, mark planned components implemented • Draft and save decisions; generate, regenerate and revert ARCH.md • Enforce Architect-only actions; read and save roles • Report workflow state and data changes to the UI	ArchitectureAgent, ConstraintElicitationAgent, ComplianceAgent, PreCheckAgent, RetrievalAgent, DiffSummarizer, DecisionService, CodebaseSnapshot, LLMClient, FileStore, AdrStore, AuditLog, AccessControl

DecisionService	• Single entry point for every new ADR • Accept an Architect's decision immediately; queue a Developer's for approval • Approve or reject pending ADRs (a rejection needs a reason) • On acceptance: patch ARCH.md, supersede the ADR it replaces, cache its embedding, write the audit trail	AdrStore, FileStore, AuditLog, AccessControl, RetrievalAgent
-----------------	--	--

Table 5.3: CRC cards, coordination layer

Class	Responsibilities	Collaborators
ArchitectureAgent	• Generate the blueprint from a system description (every component planned) • Regenerate it from a codebase snapshot, keeping every constraint and taking component status from the code • Validate the model's JSON	LLMClient
ConstraintElicitationAgent	• Turn a wizard answer into an ADR draft and at most one follow-up question • Draft an ADR from a free-text decision; suggest the ADR it supersedes; flag tentative input	LLMClient, ElicitationSession
ElicitationSession	• Walk the five topic questions and their follow-ups • Hold a topic's draft until its follow-up is answered, so each topic yields at most one ADR	none
ComplianceAgent	• Pass 1: find violations of constraints and relevant ADRs • Pass 2: sort new code into planned components now in code, extensions (at most five) and checked files with reasons • Check the model's answer against the real components; fall back to safe defaults on unparseable output	LLMClient
PreCheckAgent	• Check a coding prompt against constraints and ADRs before any code exists • Suggest a revised prompt when it conflicts	LLMClient
RetrievalAgent	• Rank accepted ADRs by relevance: TF-IDF blended with local embeddings, plus a component-name boost • Build a search query from a diff summary; cache ADR embeddings	EmbeddingService, AdrStore

DiffSummarizer	• Collect the uncommitted changes of the workspace folder (git diff and untracked files); say why a diff is empty • Skip lock files and Blueprint's own files; classify files as added, modified, renamed or deleted • Extract new imports, declarations and added or removed dependencies; build a diff excerpt	TreeSitterExtractor
----------------	--	---------------------

Table 5.4: CRC cards, agent layer

Class	Responsibilities	Collaborators
LLMClient	• Send prompts to Gemini, Groq, OpenRouter, Anthropic or OpenAI • Time out, and retry on rate limits and server errors • Validate an API key with one request; keep it in VS Code secret storage • Choose the model (setting, or a maintained default)	external LLM APIs
EmbeddingService	• Compute sentence embeddings locally (all-MiniLM-L6-v2, 8-bit) • Download the model on first use; report failure so retrieval falls back to TF-IDF	none
TreeSitterExtractor «module»	• Parse source files with Tree-sitter (eight languages) • Extract added imports and declaration headers; top-level mode for project overviews	none
CodebaseSnapshot «module»	• Build a compact project overview (dependency manifests, README, top-level declarations, file tree) for regenerating ARCH.md	TreeSitterExtractor
AdrStore	• Persist ADRs as an index plus one Markdown file each • Assign sequential IDs with serialized writes; update ADRs, renaming the file when the title changes	none
FileStore	• Read and write ARCH.md and arch.json • Apply targeted patches that keep manual edits (add component, add or replace constraint, mark implemented) • Snapshot ARCH.md before every change; revert to a snapshot	none

AuditLog	• Append audit events: reviews, pre-checks, decisions, ARCH.md and role changes • Notify listeners on every append	none
AccessControl	• Identify the user from git user.email • Read and write roles.json; resolve Architect or Developer	none

Table 5.5: CRC cards, infrastructure layer

5.4 Persistent Data Sources

BluePrint uses no database. Everything a team shares is stored as JSON and Markdown files inside the project, so architectural decisions travel with the code through git and can be reviewed in pull requests like any other change. Per-user data (the API key, settings and the embedding model) stays in VS Code's own storage and never enters the repository.

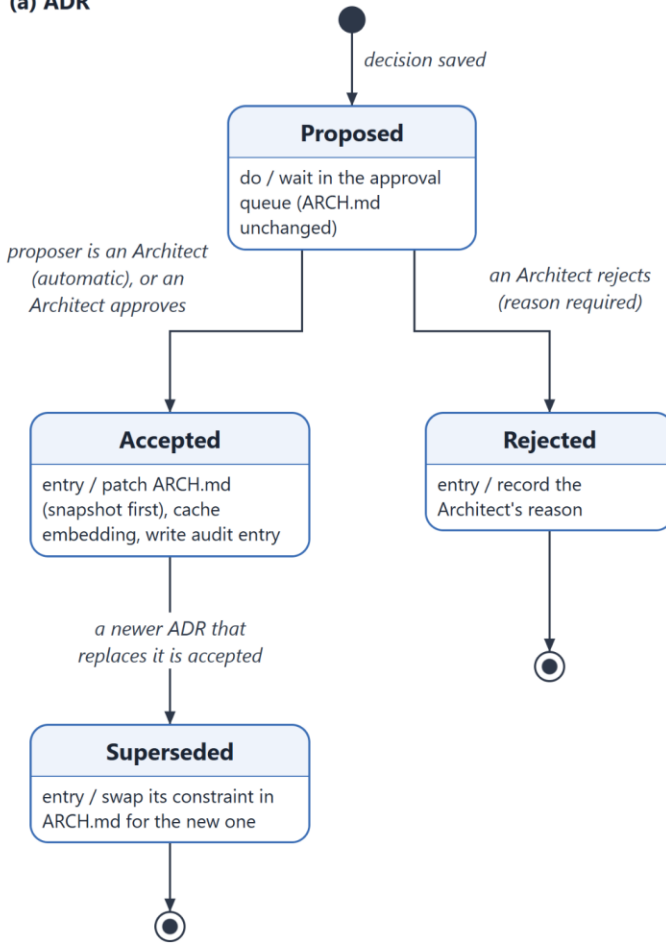
5.5 Behavioural Representations

This section elaborates the behaviour of the components with state diagrams for the classes whose objects have a lifecycle, a sequence diagram for the main use case, and an event-driven collaboration diagram for the whole extension.

5.5.1 State Diagrams

Figure 5.4 (a) shows the lifecycle of an ADR. Every decision, whichever way it is made, is first recorded as **proposed**. It is accepted at once when the proposer is an Architect; otherwise it waits in the approval queue until an Architect approves or rejects it. Only accepted ADRs change ARCH.md and are used as context in later checks. An accepted ADR becomes **superseded** when a newer ADR that replaces it is accepted. Figure 5.4 (b) shows the status of an architecture component: planned components become implemented when their code appears, without a new ADR.

(a) ADR



(b) ArchComponent

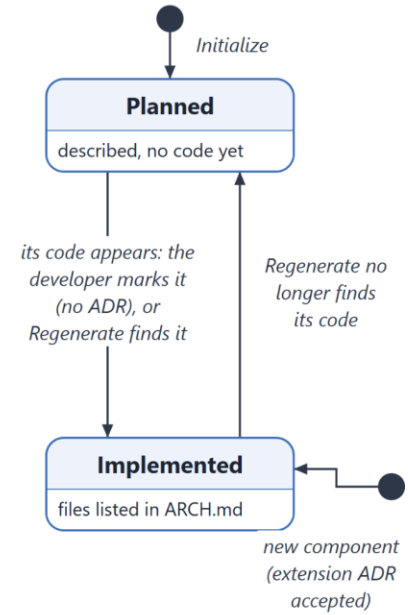


Figure 5.4: State diagrams of (a) an ADR and (b) an architecture component

Figure 5.5 shows the states of a compliance review as the developer sees them in the status bar. A review can be requested from the Hub, from the command palette or automatically when new files are created. While the panel is open, clicking the Violation state brings the existing review back instead of running a new one; the state returns to Idle once every violation has been handled.

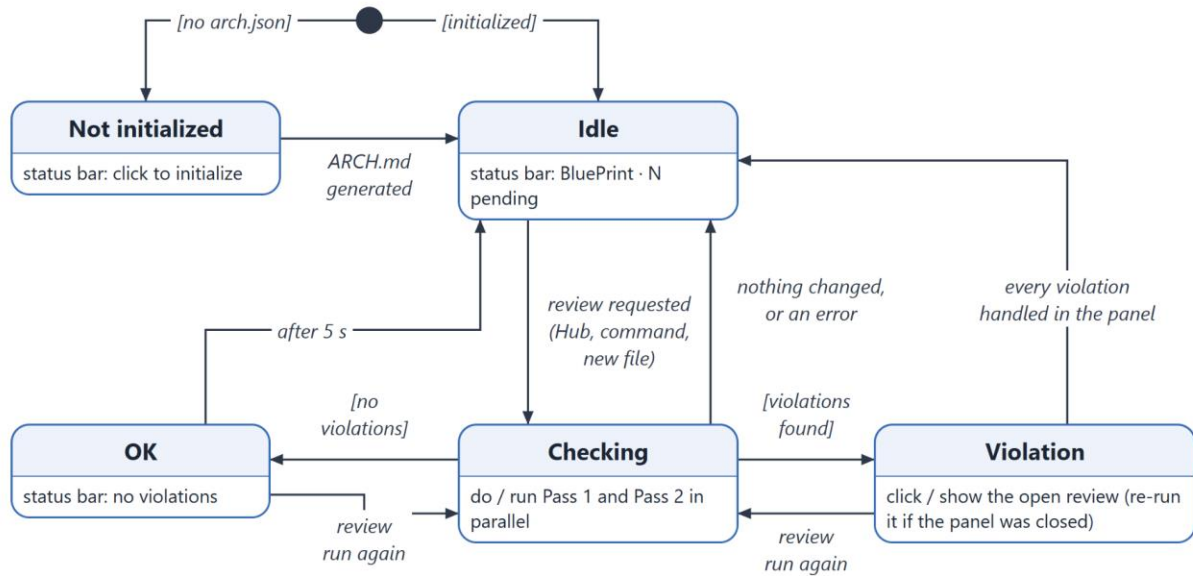


Figure 5.5: State diagram of a compliance review (status bar)

5.5.2 Sequence Diagram: Reviewing a Code Change

Figure 5.6 shows the main use case, reviewing the developer's uncommitted changes. The DiffSummarizer reduces the change to structural signals; the two compliance passes then run in parallel. Pass 1 retrieves the five most relevant accepted ADRs and asks the model for violations; Pass 2 asks the model to sort the new code into planned components, extensions and checked files. The ComplianceAgent validates both answers before they reach the panel, and the developer resolves each item individually.

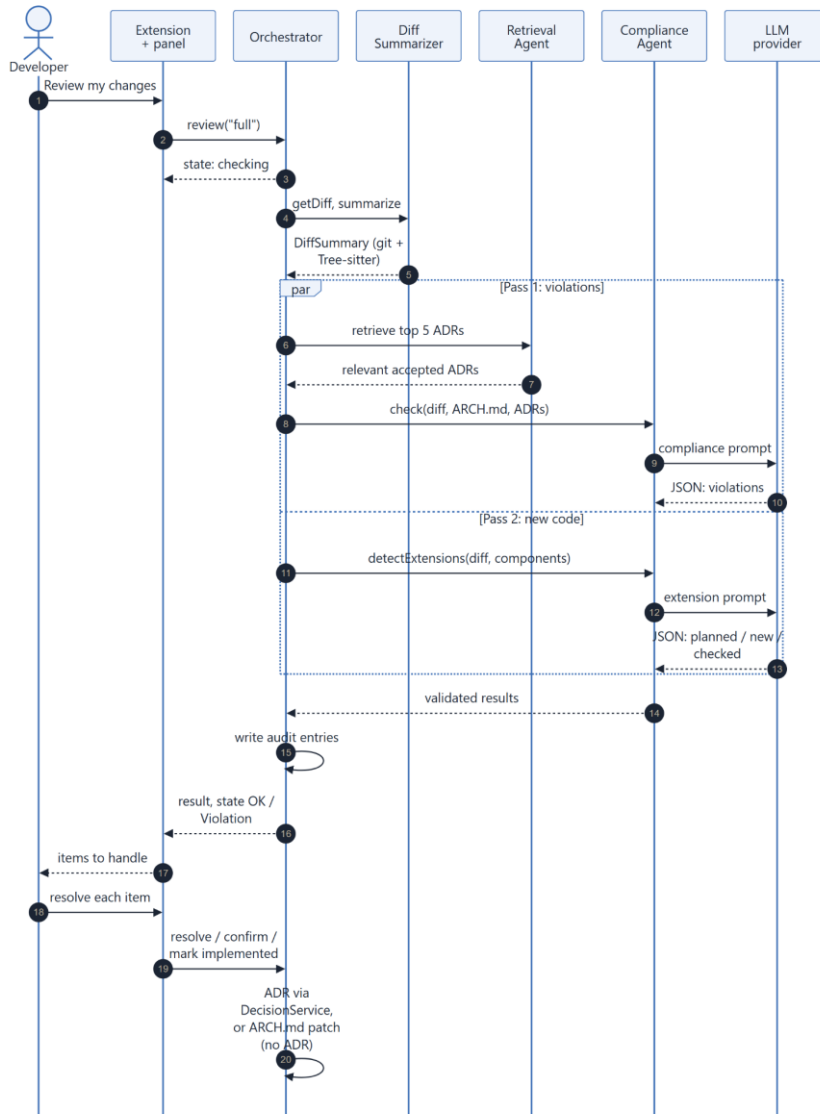


Figure 5.6: Sequence diagram for reviewing a code change

5.5.3 Event-Driven Collaboration Diagram

Figure 5.7 combines the classes of Section 5.3 with the events that make them collaborate. Events come from two sources outside the extension: the developer (commands, sidebar actions and panel forms) and the VS Code workspace (files created or changed). The Orchestrator dispatches four pipeline events (PROJECT_INIT, PROMPT_SUBMITTED, CODE_GENERATED, MANUAL_REVIEW_REQUESTED) and the decision, approval, role and regeneration requests; each agent box names the event that runs it. Changes flow back to the UI through the onState and onDataChanged hooks and the audit log's onAppend notification.

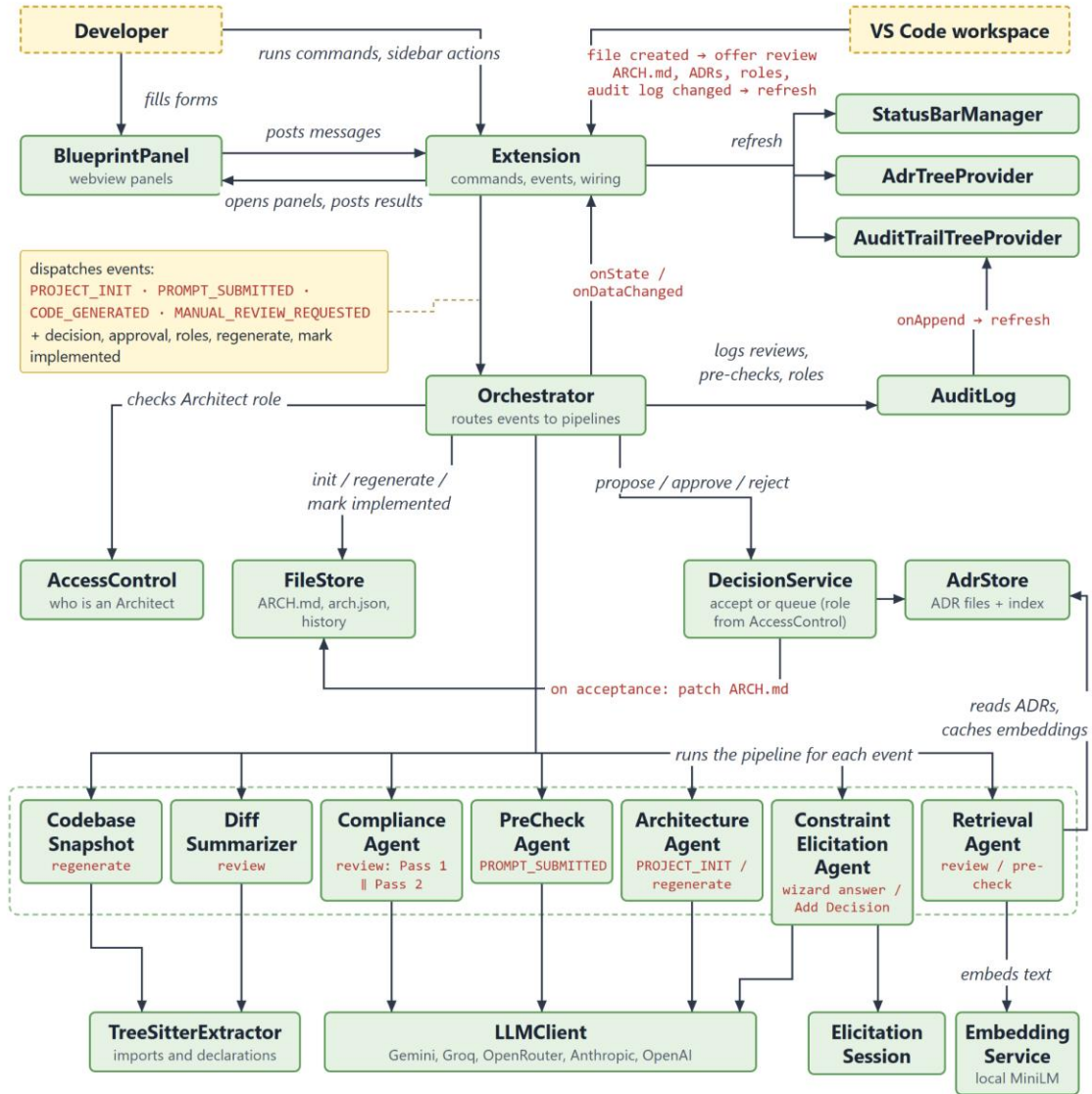


Figure 5.7: Event-driven collaboration diagram

6. Interface Design

BluePrint has no window of its own: it lives inside Visual Studio Code and uses two kinds of interface. **Native VS Code elements** (a status bar item, two sidebar views, commands in the command palette, quick picks, input boxes and notifications) give quick access and show state at a glance. **Webview panels** (HTML pages that open beside the editor) hold the longer workflows: the Hub, the Setup Wizard, the Compliance Review, the Pre-Check, Add Decision, the ARCH.md Viewer, the Audit Trail and Team & Roles. Every panel uses VS Code's theme colours, so it matches the developer's light or dark theme, and each panel type opens only once (a second request brings the open panel to the front).

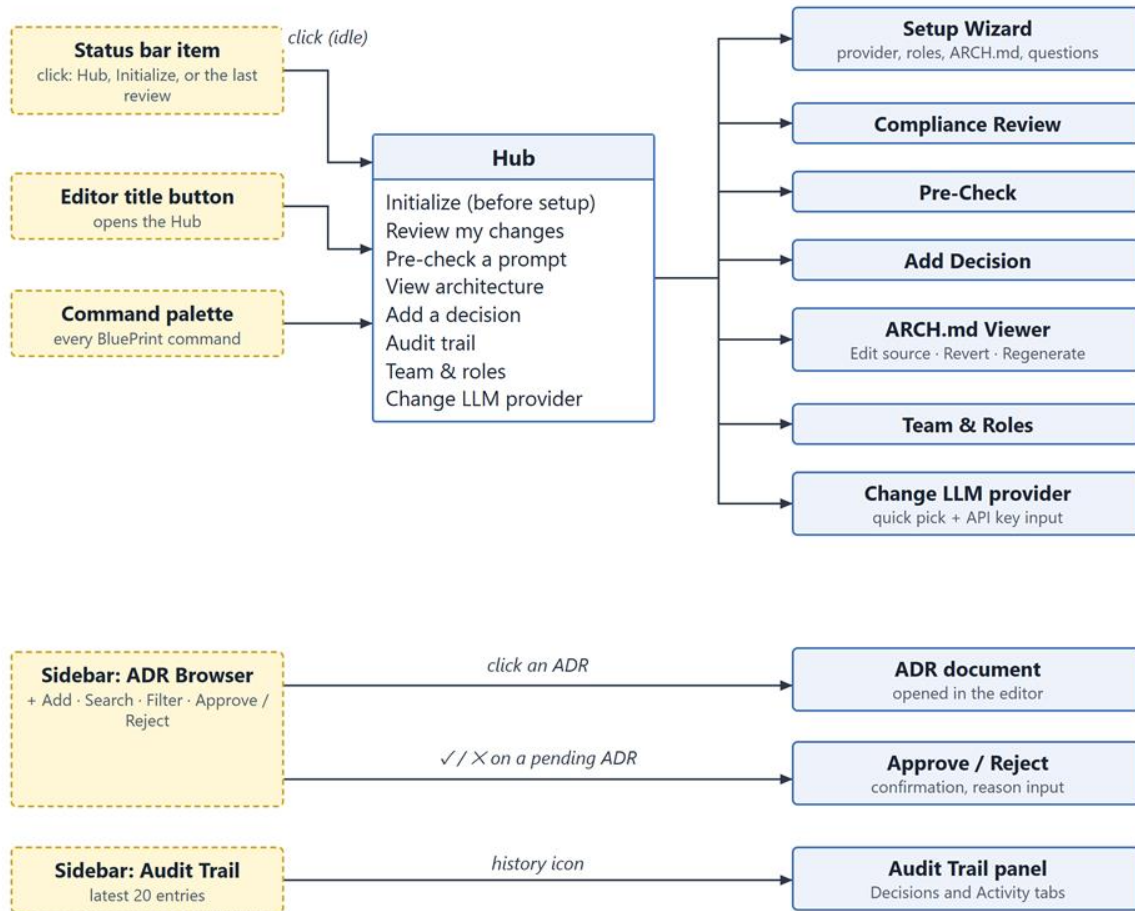
6.1 User Interface Objects and Actions

Table 6.1 lists the interface objects identified during user-interface analysis and the actions (operations) each one offers. Figure 6.1 shows how the developer moves between them: the Hub is the main entry point, and the command palette reaches every action directly.

Interface object	Kind and location	Actions (operations)
Status bar item	Native: VS Code status bar	Shows the state (not initialized, idle with pending-approval count, checking, OK, violation) and the user's role in the tooltip. Click opens the Hub, starts Initialize, or reopens the last review.
Editor title button	Native: editor title bar (initialized projects only)	Opens the Hub.
Commands	Native: command palette (BluePrint: ...)	Initialize; Open Hub; Review This Change; Pre-Check Prompt; View Architecture; Add Decision; Answer Constraint Questions; Search, Filter or Clear the ADR filter; Approve or Reject (Architects); Revert or Regenerate ARCH.md (Architects); Configure Team & Roles; Change LLM Provider / API Key; View Audit Trail.
ADR Browser	Native: sidebar tree view	Lists ADRs grouped by status, pending approvals first. Open an ADR; add a decision (+); search; filter and clear the filter; approve or reject a pending ADR (Architects).
Audit Trail (sidebar)	Native: sidebar tree view	Lists the 20 latest audit entries. Open the linked ADR or the full Audit Trail panel.
Hub	Webview panel	Initialize (before setup). Afterwards: Review my changes; Pre-check a prompt; View architecture; Add a decision; links to Audit trail, Team & roles and Change LLM provider; a banner for pending approvals opens the ADR Browser.
Setup Wizard	Webview panel	Choose the default or one's own API key; choose a provider and enter the key (validated before saving); set up team roles; describe the system; generate ARCH.md; answer, skip or analyse the constraint questions; edit, save or discard each draft ADR; open ARCH.md.

Compliance Review	Webview panel	For each violation: Update Architecture (with reasoning) or Modify Code. For each planned component now in code: Mark as Implemented, or It's a New Component Instead. For each new component: Register (with optional context) or Dismiss. For each checked file: Register as new component anyway. Re-run review, Check again, Try again.
Pre-Check	Webview panel	Enter a coding prompt; Check Architecture (or Ctrl+Enter); Use this prompt (the suggested revision); choose ADRs and Copy prompt with context; open an ADR; Revise prompt; Check another prompt.
Add Decision	Webview panel	Describe a decision in plain words; edit the drafted ADR; choose the existing decision it replaces (one may be suggested); Save ADR; edit the description; answer the guided questions instead; open the saved ADR.
ARCH.md Viewer	Webview panel	Read the rendered ARCH.md with the sections changed by the last five changes highlighted; Edit source; Revert to an earlier version and Regenerate from the codebase (both Architect-only).
Audit Trail panel	Webview panel	Switch between the Decisions and Activity tabs; search; filter by status or event group; open ADRs and changed files.
Team & Roles	Webview panel	See one's identity and role; edit the lists of Architects and Developers (Architects only); Save roles; Edit roles.json directly.
Quick picks, inputs and notifications	Native VS Code dialogs	Change the provider (provider list, then a password box with a link to the key page); pick an ARCH.md version to revert to (with a diff); confirm a regeneration (with a diff); enter an approval or rejection note; answer the new-file prompt (Review or Skip).

Table 6.1: User interface objects and their actions



The command palette reaches every panel and action directly. Before setup the status bar opens Initialize; while it shows a violation it reopens the last review.

Figure 6.1: Interface objects and the navigation between them

6.2 Events and Interface State Changes

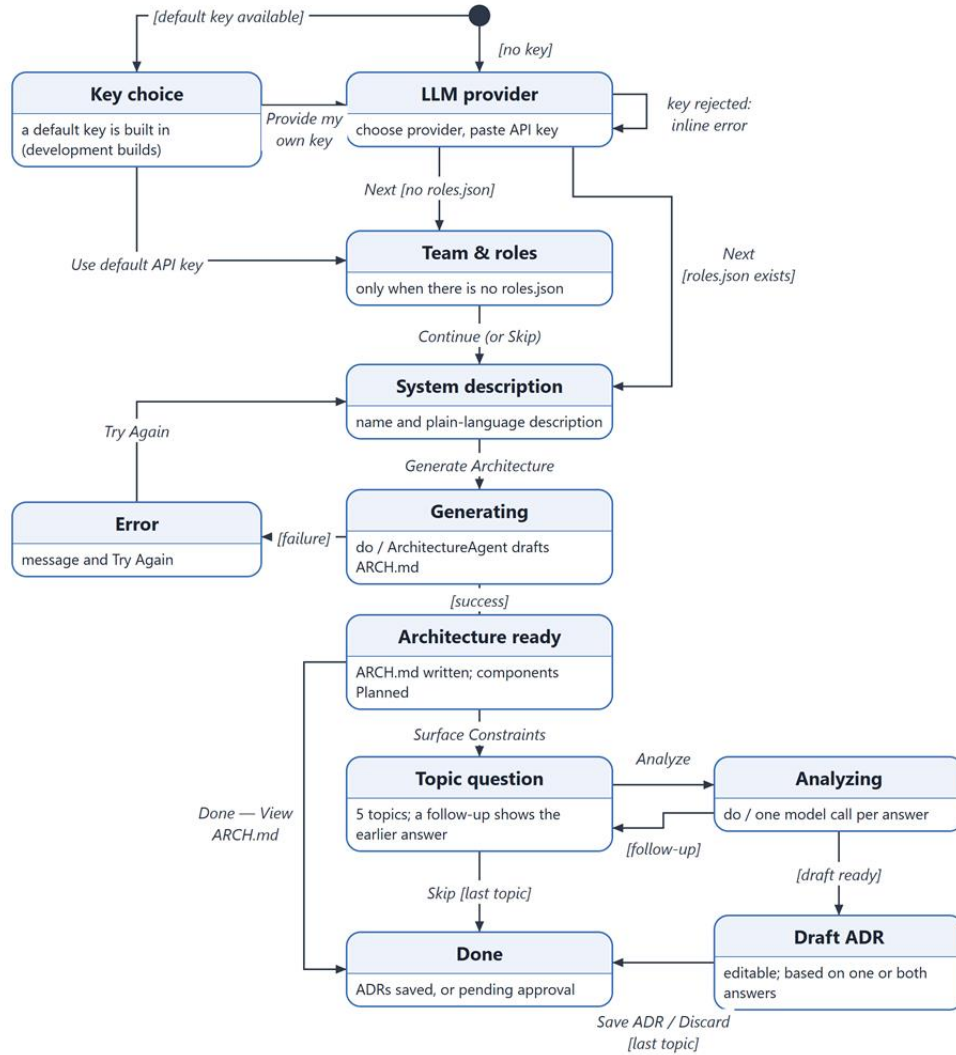
Table 6.2 lists the events that change the state of the interface. Most are user actions; the rest come from the workspace (files created or changed, for example after git pull) or from the extension when a model call finishes. The extension keeps every view in step with the files on disk, so a change made by a teammate appears without reloading.

Event	Source	Interface state change
Extension starts in a project without BluePrint	VS Code	Status bar: Not initialized. Hub shows only Initialize. Sidebar views and the editor title button are hidden.
ARCH.md generated by the wizard	Extension	Wizard moves to Architecture ready. Sidebar views and the editor title button appear; the status bar becomes Idle; an open Hub switches to its dashboard.
New file(s) created	Workspace	One notification per batch of files ("New file detected — review architectural impact?" with Review and Skip).

Review requested	Developer	Compliance Review opens on Checking; status bar shows Checking.
Review finished	Extension	Panel shows Results, Clean, No changes (with the reason) or Error. Status bar shows OK (back to Idle after 5 seconds) or Violation.
Item resolved in the review	Developer	The card is replaced by its outcome; the footer counts the items still open. When every violation is handled the status bar returns to Idle.
Status bar Violation clicked	Developer	The open review is brought to the front; if it was closed, the review runs again.
Decision saved by a Developer	Developer	The ADR appears under Pending approval ("waiting for an Architect"); the pending count appears in the status bar and the Hub banner.
Pending decision approved	Architect	The ADR moves to Accepted; ARCH.md is updated and the viewer highlights the changed section; the pending count drops.
Wizard answer analysed	Extension	The wizard shows a follow-up question (with the earlier answer above it), a draft ADR, or the next topic.
Pre-check finished	Extension	Pre-Check shows the result: concerns, a suggested revision and the ADRs to attach, or that no conflicts were found.
ADR Browser filtered	Developer	The list narrows; a banner shows the filter and a Clear filter button appears in the view title.
Files changed on disk (git pull, edited roles.json)	Workspace	Sidebar, pending count, role, initialized state and open panels refresh within about a second.
User's role becomes Developer	roles.json	Approve, Reject, Revert and Regenerate are hidden or disabled; pending ADRs read "waiting for an Architect".
Audit entry written	Extension	The sidebar Audit Trail and an open Audit Trail panel show the new entry.

Table 6.2: Events and the interface state changes they cause

Figures 6.2 and 6.3 show how these events move the two longest workflows from state to state. The states of the status bar item are shown in Figure 5.5.



After Save ADR, Discard or Skip on an earlier topic, the next topic question is shown; an answer with nothing concrete also moves on. A topic yields at most one ADR: with a follow-up, the draft waits for its answer.

Figure 6.2: State diagram of the Setup Wizard

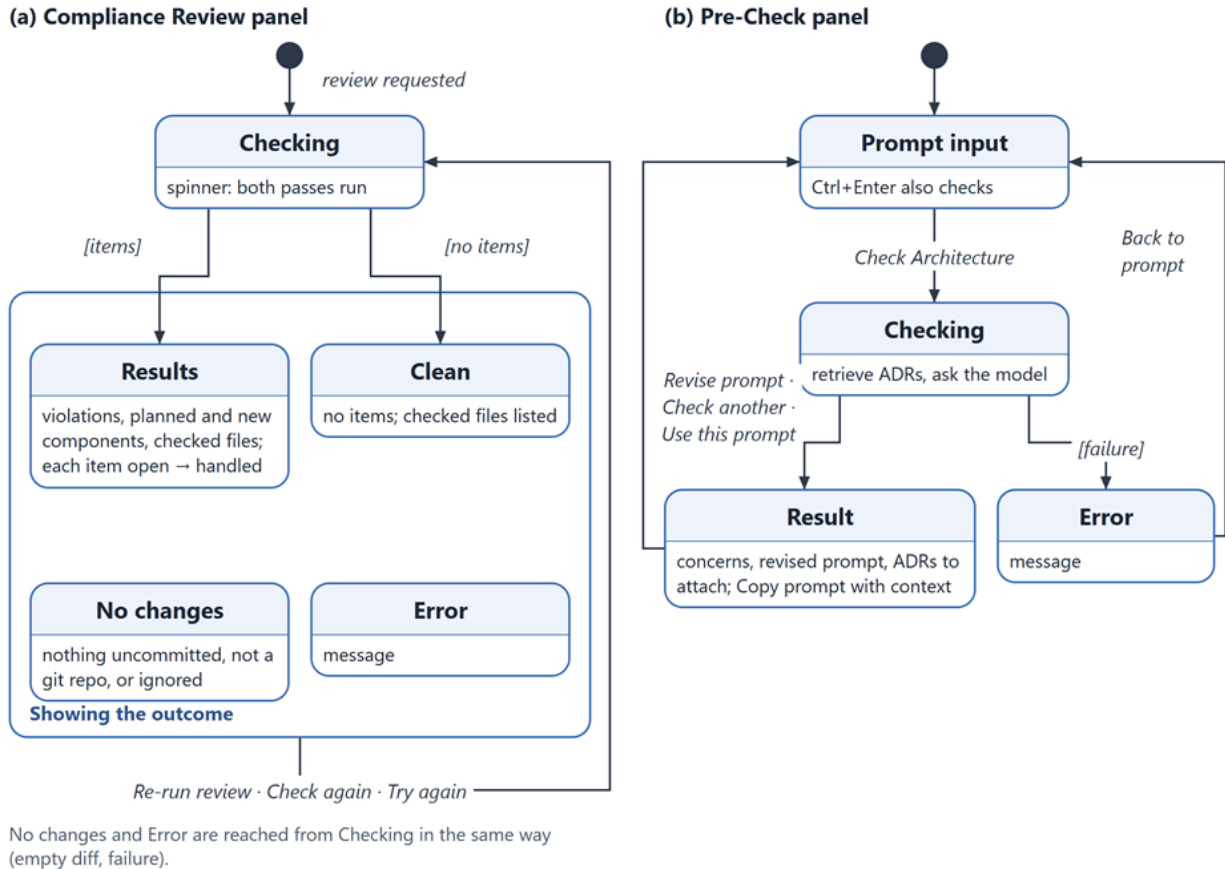


Figure 6.3: State diagrams of (a) the Compliance Review panel and (b) the Pre-Check panel

6.3 Interface States as Seen by the User

The following figures show each interface state as the developer sees it, in VS Code's default light theme. The panel images were produced from the extension's own panel pages, driven by the same messages the extension sends, with data from a sample project: Campus Eats, a cafeteria-ordering web application with a React frontend, an Express API, PostgreSQL and Stripe. The status bar and sidebar are native VS Code elements and are shown as reconstructions.

6.3.1 Status Bar and Sidebar

The status bar item is always visible and summarises Blueprint's state (Figure 6.4). The sidebar (Figure 6.5) lists the decisions, with pending approvals first and Approve and Reject buttons for Architects, and the latest activity from the audit log.

Not initialized	main Blueprint: Not initialized click: initialize
Idle	main Blueprint · 1 pending click: open the Hub
Checking	main Blueprint: Checking... review running
OK	main Blueprint: OK returns to Idle after 5 s
Violation	main Blueprint: Violation click: show the review

Figure 6.4: The status bar item in each state (reconstruction)

BLUEPRINT
...

ARCHITECTURAL DECISIONS
You: Architect
+
🔍
🔧
...

Pending approval (1)

🕒 Extension: Add Notifications component #0005 · ne... ✓ ✕

Accepted (3)

✓ Use PostgreSQL as the primary datastore #0001

✓ Authenticate students with university... #0002

✓ Serve the API from Node.js and Express #0004

Superseded (1)

➔ Process payments through Stripe #0003 · replaced by ADR-0...

AUDIT TRAIL
🕒

🕒 Planned component "Frontend" is now implemented Oct 8

🕒 ADR-0005 proposed: Extension: Add Notifications... Oct 8

📋 New component(s) detected: Notifications Oct 8

⚠️ 1 violation(s) detected Oct 8

🛡️ Pre-check: 1 concern Oct 8

✎ ARCH.md updated by ADR-0001 Oct 7

👤 Roles updated: 1 Architect, 2 Developers Oct 7

Figure 6.5: The BluePrint sidebar: ADR Browser and Audit Trail (reconstruction)

6.3.2 Hub

Before setup the Hub offers a single action, Initialize. Afterwards it shows who the user is, a banner when decisions wait for approval, the four main tasks and links to the less frequent ones (Figure 6.6).

BluePrint

Architecture-aware development assistant

GET STARTED



Initialize BluePrint

BluePrint isn't set up in this project yet. Choose an LLM provider, set up roles and generate ARCH.md for this codebase.

(a) Before initialization

BluePrint

lead@campuseats.dev · Architect

1 decision needs your approval

BEFORE YOU RUN

Reviews look at everything not yet committed: changed, new, deleted and renamed files (files ignored by `.gitignore` are skipped). Run checks **before** `git commit`.

WHAT WOULD YOU LIKE TO DO?



Review my changes

Find changes that break your constraints or decisions, and new components not yet in ARCH.md.



Pre-check a prompt

Check an AI coding prompt against your architecture before any code is written.



View architecture

Read ARCH.md with recent changes highlighted; edit, revert or regenerate it.



Add a decision

Describe a decision or a changed requirement in your own words. BluePrint drafts the ADR and updates ARCH.md.

[Audit trail](#) — every review, decision and ARCH.md change

[Team & roles](#) — you are an Architect

[Change LLM provider / API key](#)

(b) After initialization

Figure 6.6: The Hub

6.3.3 Setup Wizard

The wizard takes the developer from an empty project to an ARCH.md and the first ADRs. The API key is checked with one request before it is saved; the team-and-roles step appears only when the project has no roles file yet (Figure 6.7). The constraint questions follow: a follow-up repeats the earlier answer, and a single draft ADR based on both answers is offered for editing (Figure 6.8).

Configure LLM Provider

Choose your provider and enter your API key. It is stored securely in VSCode's secret store — never in your project files.

Provider

Groq (free tier available) ▼

API Key

.....

Free keys are available and take about a minute to create. [Get a key](#)

Next →

(a) LLM provider and API key

Team & Roles

Architects approve decisions, and their own decisions update ARCH.md straight away. Everyone else is a **Developer**: their decisions wait for an Architect's approval.

You: **lead@campuseats.dev** (from git config user.email)

☐ Just me
You become the only Architect. Anyone who joins later is a Developer until you add them.

☒ A team
You are an Architect. Add the other Architects, and Developers if you like.

☐ Skip for now
No roles: everyone is an Architect. Set them up later with BluePrint: Configure Roles.

Other Architects

lead@company.com

Developers (optional, for the record)

dev1@campuseats.dev
dev2@campuseats.dev

One email per line, matching each person's git config user.email.
Anyone not listed as an Architect is a Developer anyway.

Saved to .blueprint/roles.json. Commit it so the whole team shares it. Roles are an agreement, not a lock: anyone who can push can edit that file, so protect it with code review (for example a CODEOWNERS rule).

← Back Continue →

(b) Team and roles

Figure 6.7: Setup Wizard, first steps

Surface Constraints

Answer a few questions. Blueprint will turn concrete decisions into Architectural Decision Records you can review before saving.

Topic 1 of 5 · follow-up

FOLLOW-UP Based on your answer: "React frontend, Node.js and Express backend, PostgreSQL for all data."

Is PostgreSQL the only datastore, or are caches or search engines allowed too?

Redis is allowed, but only as a cache. No other databases.

Skip Analyze →

(a) A follow-up question

Surface Constraints

Answer a few questions. Blueprint will turn concrete decisions into Architectural Decision Records you can review before saving.

Topic 1 of 5

Based on both of your answers. Review and edit the draft ADR before saving.

Title

Use PostgreSQL as the primary datastore

Context

Campus Eats stores users, menus, orders and payment records. One relational store keeps the data consistent and simple to operate.

Decision

All persistent data is stored in PostgreSQL; Redis may be used only as a cache. No other databases.

Consequences

New features must model their data in PostgreSQL. Reviews flag any other datastore as a violation.

Discard Save ADR

(b) The draft ADR from both answers

Figure 6.8: Setup Wizard, constraint questions

6.3.4 Compliance Review

Figures 6.9 and 6.10 show one review of the sample project's uncommitted changes. A violation (orders written to MongoDB despite the PostgreSQL decision) is resolved by updating the architecture or modifying the code. The new React application is recognised as the planned Frontend and can be marked implemented without an ADR. A standalone notifications module is a new component, and a hello-world script is listed as checked with its reason.

Compliance Review

Found 1 violation and 1 planned component now in code and 1 new component. Handle each one below.

► What was checked (11 items)

Violations 1

Resolve each violation on its own. "Update Architecture" records one ADR per violation. "Modify Code" records nothing; revise the code and re-run the review.

HIGH ADR-0001

Orders are now written to MongoDB, but ADR-0001 makes PostgreSQL the only persistent datastore (Redis is allowed only as a cache).

```
src/db/orders.ts, saveOrder()
```

☐ Update Architecture
The code is right. Record an ADR and update ARCH.md (may need Architect approval).

☐ Modify Code
The constraint is right. No ADR; revise the code and re-run the review.

Reasoning (required for Update Architecture)...

Record

Planned components now in code 1

ARCH.md lists these as planned, and code for them appeared. Marking one implemented updates ARCH.md without an ADR, since it was already decided. If the code is really something else, register it as a new component instead.

PLANNED → IN CODE Frontend

Code	src/App.tsx
Why matched	A React UI for browsing the menu and placing orders: the planned Frontend's main responsibility.

Mark as Implemented

It's a New Component Instead

Figure 6.9: Compliance Review results: violation and planned component

New components 1

Structural elements not present in ARCH.md. Registering one creates its own ADR and adds it to ARCH.md. Dismissing records nothing.

NEW COMPONENT

Notifications

Responsibility	Asks for permission and shows browser notifications when an order is ready.
Technology	Web Notifications API
Code	src/notifications.ts
Why detected	A standalone notification module; no component in ARCH.md owns notifications as its own responsibility.

Detected in code that also has violations. If you revise that code, this component may no longer apply.

Why is this a new component? Context for the ADR (optional)...

Register as New ComponentDismiss

► Checked, not new (1 file)

Re-run review3 items still open.

Figure 6.10: Compliance Review results (continued): new component and checked files

When nothing needs attention the panel says so and still lists the files it checked; when it finds no changes it explains why, for example that the folder is ignored by an enclosing git repository (Figure 6.11).



No violations or new components

Changes are consistent with your architecture and ADRs.

▼ What was checked (2 items)

ADDED FILES

src/MenuFilter.tsx

NEW DECLARATIONS

```
export function MenuFilter({ tags, onChange }: Props)
```

▼ Checked, not new (1 file)

src/MenuFilter.tsx

Adds a UI filter to the already implemented Frontend.

[Register as new component anyway](#)

(a) No violations or new components



Git is ignoring this folder

This folder is inside the git repository at `D:\projects\monorepo`, whose ignore rules hide it, so

Blueprint cannot see its changes.

Run `git init` in this folder to give it its own repository, then check again.

Check again

(b) No changes found, with the reason

Figure 6.11: Compliance Review outcomes without items

6.3.5 Pre-Check and Add Decision

The Pre-Check panel checks a prompt for an AI coding assistant before any code is written. In Figure 6.12 the prompt asks for a MongoDB collection; the panel cites ADR-0001, suggests a revised prompt, and offers to copy the prompt together with the relevant ADRs. Add Decision (Figure 6.13) turns a plain-language decision into an editable ADR and suggests the existing decision it replaces.

Pre-Flight Concerns

1 concern

These concerns are based on your stated intent. No code has been written yet.

HIGH

ADR-0001

The prompt asks for a MongoDB collection, but ADR-0001 makes PostgreSQL the only persistent datastore.

SUGGESTION

Store favourites in a PostgreSQL table instead.

Suggested revision

Your prompt, adjusted to stay within the architecture.

Add a favourite_dishes table in PostgreSQL (user_id, dish_id, created_at) and an Express endpoint to read and update a student's favourites.

Use this prompt

Attach architectural context

Copy your prompt together with the selected ADRs and the ARCH.md constraints, then paste it into your AI assistant.

☒ ADR-0001 Use PostgreSQL as the primary datastore

cited by a concern

Open

☒ ADR-0004 Serve the API from a Node.js and Express backend

relevant

Open

► Less relevant (1)

Copy prompt with context

Revise prompt

Check another prompt

Figure 6.12: Pre-Check result for a conflicting prompt

Review the draft

Edit anything before saving. The title becomes the constraint line in ARCH.md.

Title

Accept payments through Stripe and bKash

Context

Many students pay with bKash. Offering it next to cards increases online orders.

Decision

Checkout offers two payment providers: Stripe for cards and bKash for mobile wallets.

Consequences

The Payment Service integrates both providers. ADR-0003 (Stripe only) is replaced.

Replaces an existing decision?

ADR-0003: Process payments through Stripe (suggested)

Why: ADR-0003 allows Stripe only; this decision adds bKash as a second provider.

Its decision: Card payments go through Stripe only.

On saving, it is marked superseded and its constraint in ARCH.md is replaced by this one.

As an Architect, saving accepts this decision and updates ARCH.md straight away, replacing ADR-0003's constraint.

← Edit description

Save ADR

Figure 6.13: Add Decision with a suggested replacement

6.3.6 ARCH.md Viewer, Audit Trail and Team & Roles

The ARCH.md Viewer (Figure 6.14) renders the architecture and highlights the sections changed by recent decisions, with the reason for each change; the Components table shows which components are planned and which are implemented. The Audit Trail (Figure 6.15) records every review, decision and change, searchable and filterable. Team & Roles (Figure 6.16) shows the user's role and lets Architects edit the team.

Architecture: Campus Eats

Last updated: 08/10/2026, 16:05

System Overview

Campus Eats lets students browse the cafeteria menu, order and pay online, and get notified when their order is ready. Staff manage the menu and order status from a dashboard.

Components

- Updated 4 min ago · ADR-0005: Extension: Add Notifications component (+1 earlier)

Component	Responsibility	Technology	Status
Frontend	Student and staff web interface	React, TypeScript	Implemented — <code>src/App.tsx</code>
Backend API	REST endpoints for menu, orders and payments	Node.js, Express	Planned
Database	Users, menus, orders and payments	PostgreSQL	Planned
Payment Service	Processes card payments	Stripe	Planned
Notifications	Browser notifications when an order is ready	Web Notifications API	Implemented — <code>src/notifications.ts</code>

Data Flow

The React frontend calls the Express API, which reads and writes PostgreSQL and calls Stripe for payments.

Constraints

- Updated 1 d ago · ADR-0001: Use PostgreSQL as the primary datastore
 - All persistent data is stored in PostgreSQL; Redis may be used only as a cache.
 - Sign-in uses Google OAuth restricted to the university domain.
 - Card payments go through Stripe only.

Open Questions

- Should staff and students use separate front-end applications?

Figure 6.14: ARCH.md Viewer with recent changes highlighted

Decision Audit Trail

Decisions 5 Activity 8

Search titles, decisions, files, people... All Decisions Reviews Pre-checks ARCH.md

✓ COMPONENT IMPLEMENTED lead@campuseats.dev Oct 8, 2026, 05:50 PM

Planned component "Frontend" is now implemented (src/App.tsx)

src/App.tsx

✓ ADR APPROVED lead@campuseats.dev Oct 8, 2026, 05:49 PM ADR-0005

ADR-0005 approved

✗ ADR PROPOSED dev1@campuseats.dev Oct 8, 2026, 05:44 PM ADR-0005

ADR-0005 proposed: Extension: Add Notifications component

◇ NEW COMPONENT dev1@campuseats.dev Oct 8, 2026, 05:43 PM

New component(s) detected: Notifications

src/notifications.ts

🚩 COMPLIANCE REVIEW dev1@campuseats.dev Oct 8, 2026, 05:43 PM

1 violation(s) detected

src/notifications.ts src/App.tsx src/db/orders.ts hello.py

🛡️ PRE-CHECK dev2@campuseats.dev Oct 8, 2026, 04:58 PM

Pre-check: 1 concern

✎ ARCH.MD UPDATED lead@campuseats.dev Oct 7, 2026, 03:53 PM ADR-0001

ARCH.md updated by ADR-0001

👤 ROLES UPDATED lead@campuseats.dev Oct 7, 2026, 02:53 PM

Roles updated: 1 Architect(s) (lead@campuseats.dev), 2 Developer(s)

Figure 6.15: Audit Trail panel, Activity tab

Team & Roles

Architects approve decisions, and their own decisions update ARCH.md straight away.
Everyone else is a **Developer**: their decisions wait for an Architect's approval.

You: **lead@campuseats.dev** **ARCHITECT**

Architects

lead@campuseats.dev (you)

Other Architects, one email per line

Developers (optional, for the record)

dev1@campuseats.dev
dev2@campuseats.dev

One email per line, matching each person's `git config user.email`. Anyone not listed as an Architect is a Developer anyway.

Stored in `.blueprint/roles.json`. Commit it so the whole team shares it. Roles are an agreement, not a lock: anyone who can push can edit that file, so protect it with code review (for example a CODEOWNERS rule).

Save roles

[Edit roles.json directly](#)

Figure 6.16: Team & Roles panel

7. Testing

This section describes how BluePrint was tested: the strategy, the criteria for passing or failing a test item, the main risks and how they are handled, and the main test cases with their outcomes. Test case identifiers in the form T_n refer to the test plan of the Software Requirements Specification (SRS).

7.1 Testing Approach and Strategy

Testing combines four levels, from fast and deterministic to slow and realistic:

- **Unit and integration tests** run with Vitest on every change: 169 tests in 8 files covering the LLM client (routing for five providers, retries, key validation), the agents, diff summarization, retrieval, ADR storage, ARCH.md patching and history, roles and the approval queue, and the UI state classes. The VS Code API is replaced by a stand-in backed by real temporary folders, so file reading and writing are exercised for real; the language model is replaced by a scripted fake that returns prepared answers and records each prompt.
- **Integration with real tools.** Diff tests create temporary git repositories (commits, renames, deletions, untracked files, nested and ignored folders) and parse the changes with the real Tree-sitter grammars.
- **Live model tests.** Behaviour that depends on the model's judgment (the wizard's follow-ups, extension detection) is checked against a real provider (Groq, model openai/gpt-oss-120b) on the sample projects, several runs per case, because model output varies between runs.
- **Manual acceptance tests** in the VS Code Extension Development Host follow a checklist that maps every SRS test case (T1 to T53) to an automated test or a manual step.

Static checks (TypeScript type checking and ESLint) run alongside the tests. When a defect is fixed, a regression test is added, and for important fixes the test is confirmed to fail when the fix is removed (a manual mutation check), so the test is known to catch the defect.

7.2 Item Pass/Fail Criteria

- **Build:** a change is accepted only when every automated test passes, the type check reports no errors and the linter reports no problems.
- **Automated test case:** passes when all its assertions hold; any failed assertion or unexpected exception fails it.
- **Model-dependent behaviour:** passes when the answer is parsed into a valid result and the expected outcome is produced in every one of four live runs; one wrong run fails it and the prompt is revised.
- **Manual test case:** passes when the observed behaviour matches the expected result in the checklist; a crash, an unhandled error, lost data or a blocked interface fails it.
- **Robustness:** malformed model output, network failures and missing files must never crash the extension or corrupt stored data; the user must see a clear message instead.

7.3 Risks and Contingencies

Risk	Impact	Contingency
Model answers vary between runs	Inconsistent violations or component verdicts	Strict JSON format, answers checked against the real components, safe defaults; every verdict can be overruled in the panel; repeated live runs before a prompt change is accepted

Provider rate limits or outages	Checks fail or slow down	Time-out and retry with back-off; clear error with Try again; provider can be changed without re-initializing
Provider retires a model	Requests fail (HTTP 404)	Maintained default models, a per-provider model setting, and an error message that names the setting
Embedding model unavailable (offline, unsupported platform)	Less accurate ADR retrieval	Automatic fallback to lexical (TF-IDF) ranking, covered by tests
Platform-specific packages not run on Linux or macOS	Installation problems on those systems	A universal package without native code; marked as untested
Project not a git repository, or inside a larger one	Reviews find nothing to check	The panel states the reason and the fix; a non-git fallback is planned
Concurrent writes to the ADR index	Duplicate or skipped ADR numbers	Writes are serialized; covered by a concurrency test
Manual acceptance testing not yet complete	Interface defects may remain	Checklist in progress; each defect found so far was fixed with a regression test

Table 7.1: Testing risks and contingencies

7.4 Test Cases

Table 7.2 lists the main functional and API test cases. "Auto" cases run in the automated suite; "Live" cases were run against the real model; "Manual" cases are part of the acceptance checklist.

ID	Feature (SRS)	Input / procedure	Expected outcome	Actual outcome	Result
TC-01	LLM request routing (T7–T9)	Send a prompt through each provider: Gemini, Anthropic, OpenAI, Groq, OpenRouter (Auto)	Correct endpoint, model and message format; API key sent in a header, never in the URL	Requests built as expected for all five providers	Pass
TC-02	LLM errors and retries (T10)	Provider returns 429 (with retry-after), a non-retryable error, a 404, and an empty response (Auto)	429 retried with back-off, giving up after three attempts; other errors reported with their status; a 404 names the model setting; an empty response is an error, not a crash	As expected	Pass

TC-03	API key storage and validation (T5, T6)	Save a key; validate a working key, a rejected key, an unknown model and a blocked project (Auto, Live)	Key and provider stored only in secret storage; each failure mapped to an actionable message; a working key accepted	As expected; a real Groq key validated live	Pass
TC-04	Architecture generation (T11, T13, T14)	Model returns valid JSON, fenced JSON, and plain text (Auto)	Blueprint written to arch.json and ARCH.md with every component Planned; fences stripped; plain text reported with the raw response	As expected	Pass
TC-05	Constraint questions (T16–T18)	Concrete, tentative and empty answers (Auto)	Draft ADR for a concrete answer; none for a tentative one; no model call for an empty one	As expected	Pass
TC-06	One ADR per topic (follow-ups)	Topic answer with a follow-up, then answer, skip, or answer "not sure" (Auto, Live)	Draft held until the follow-up; one ADR covering both answers; the first answer's draft offered on skip	As expected; 3 of 3 live topics gave one combined draft keeping facts from both answers	Pass
TC-07	Diff summarization (T21–T25)	Git repository with modified, new, renamed, deleted, empty and untracked files; nested and ignored folders (Auto)	Files classified correctly; new imports, declarations and dependency changes extracted; only the workspace's own changes; the reason given when nothing is found	As expected	Pass
TC-08	ADR retrieval (T26–T31)	Several accepted ADRs and a query; embedding model unavailable (Auto)	Top 5 ranked by blended score with component-name boost; embeddings cached; lexical fallback	As expected	Pass
TC-09	Violation detection (T32, T35)	Model reports a violation; model returns malformed output (Auto)	Violation shown with severity and location; malformed output treated as no violation, without crashing	As expected	Pass

TC-10	Planned vs. new components (T37)	Sample projects: a payment module in a PDF tool; a notifications module; a React app for a planned Frontend; a file next to the implemented Frontend (Live, 4 runs each)	New component; new component; planned component now in code; checked, not new	4 of 4 runs correct for each case	Pass
TC-11	Resolving review items (T36, T39)	Update Architecture on two violations; register an extension (Auto)	One ADR per item; the extension's ARCH.md row is added as Implemented with its files on acceptance	As expected	Pass
TC-12	Mark a planned component implemented	Mark the planned Frontend implemented as a Developer (Auto)	ARCH.md and arch.json updated, no ADR created, history snapshot and audit entry written	As expected	Pass
TC-13	Roles and approval queue (SRS 3.2.5)	Developer saves a decision; Architect approves or rejects; Developer tries to approve, revert or initialize (Auto)	Developer's ADR pending and ARCH.md unchanged until approval; rejection needs a reason; Architect-only actions refused for Developers	As expected	Pass
TC-14	ADR storage (T41–T43)	Create ADRs concurrently; change an ADR's title (Auto)	Sequential zero-padded IDs with no duplicates; file renamed with the title; index and Markdown updated	As expected	Pass
TC-15	ARCH.md patching and history	Apply ADR effects to a hand-edited ARCH.md; revert; older table without a Status column (Auto)	Manual edits preserved; snapshot before every change; revert restores; Status column added	As expected	Pass
TC-16	Prompt pre-check (T45, T46)	Prompt that conflicts with an ADR; prompt without conflicts; copy with context (Auto)	Concerns with a suggested revision; clean result; copied text contains the prompt, the selected ADRs and the constraints	As expected	Pass

TC-17	Replacing a decision (Add Decision)	Architect saves a decision that replaces an accepted ADR; Developer does the same (Auto)	Old ADR superseded and its ARCH.md constraint swapped; for a Developer, nothing changes until approval	As expected	Pass
TC-18	Status bar behaviour (T4)	Violation found, clicked, then every violation handled (Auto)	Click reopens the review; the state returns to Idle when every violation is handled	As expected	Pass
TC-19	Extension in VS Code (T1–T3, T44, T47–T53)	Manual checklist in the Extension Development Host on the sample projects	Activation, sidebar, panels, singleton panels and the audit-trail handshake behave as specified	Partly run: three defects found and fixed with regression tests;	Pass

Table 7.2: Test cases with outcomes

All 169 automated tests pass, and the type check and linter report no problems. Every live case met its criterion of four correct runs out of four. The manual acceptance checklist was partly run: it found three defects (reviews of nested folders saw no changes, the wizard created near-duplicate ADRs from follow-up questions, and a new component was missed because planned and implemented components were not told apart), each of which was fixed and covered by new tests.

8. User Manual

This section explains how to install BluePrint, set it up for a project and use it day to day. It is written for developers who already use Visual Studio Code (VS Code). Screenshots of every panel mentioned here are in Section 6.3; the figure numbers are given in brackets.

8.1 System Requirements

- **Visual Studio Code** version 1.85 or later.
- **Operating system:** Windows (x64 or ARM64), Linux (x64 or ARM64) or macOS on Apple silicon. Other systems, such as macOS on Intel, use the universal package (Section 8.2).
- **Git**, with the project folder initialised as a git repository. Compliance reviews compare the project with its last commit, so they find nothing to check without git.
- **An API key** for one of the supported language-model providers: Google Gemini, Groq, OpenRouter, Anthropic or OpenAI (Section 8.3). Gemini, Groq and OpenRouter offer free tiers.
- **An internet connection** for the language-model calls. On first use BluePrint also downloads a small embedding model (about 23 MB) for ranking ADRs; if this is not possible it falls back to word-based ranking automatically.
- For multi-person projects, each team member's git config user.email should be set, since roles are assigned by e-mail address.

8.2 Installation

BluePrint is distributed as a VS Code extension package (a .vsix file). One package is built for each platform, because each carries the native runtime of the local embedding model for that platform only; the universal package has no native code and works everywhere, ranking ADRs by words alone. Choose the package from Table 8.1.

Package	Platform	Notes
blueprint-win32-x64.vsix	Windows, Intel or AMD 64-bit	Most Windows PCs
blueprint-win32-arm64.vsix	Windows on ARM	
blueprint-linux-x64.vsix	Linux, 64-bit	
blueprint-linux-arm64.vsix	Linux on ARM	
blueprint-darwin-arm64.vsix	macOS on Apple silicon (M1 and later)	
blueprint-universal.vsix	Any platform	No local embedding model; ADRs are ranked by words only

Table 8.1: Installation packages

8.2.1 Installing a package

To install from within VS Code:

1. Open the Extensions view (Ctrl+Shift+X, or Cmd+Shift+X on macOS).

2. Click the ... (Views and More Actions) menu at the top of the view and choose **Install from VSIX....**
3. Select the downloaded .vsix file and confirm. Reload VS Code if it asks.

Alternatively, install from a terminal with code `--install-extension blueprint-win32-x64.vsix` (using the file name of the chosen package).

8.2.2 Building from source

Developers who want to build the packages themselves need Node.js 20 or later and npm:

1. Clone the repository and run `npm install` in its root folder.
2. Run `npm run package` to build the package for the current platform, or `npm run package:all` for every platform. The packages are written to the `out/` folder.
3. Install the package as in Section 8.2.1. To try the extension without installing it, open the repository in VS Code and press F5; this starts an Extension Development Host window with BluePrint loaded.

`npm test` runs the automated tests, and `npm run typecheck` and `npm run lint` run the static checks.

8.2.3 Checking the installation

After installation, a BluePrint icon appears in the Activity Bar on the left of the window, and the status bar at the bottom reads **BluePrint: Not initialized** whenever a folder is open (Figure 6.4). Clicking that item opens the Hub, from which the project is initialised.

8.3 Obtaining an API Key

BluePrint sends its requests to a language-model provider using the developer's own API key. Create a key at one of the pages in Table 8.2; the setup wizard also links to the right page for the chosen provider. The key is stored in VS Code's encrypted secret storage on the developer's machine, never in the project's files, so each team member uses their own key.

Provider	Where to create a key	Default model	Free tier
Google Gemini	aistudio.google.com/apikey	gemini-flash-latest	Yes
Groq	console.groq.com/keys	openai/gpt-oss-120b	Yes
OpenRouter	openrouter.ai/keys	openrouter/free	Yes (free models)
Anthropic	console.anthropic.com/settings/keys	claude-sonnet-4-6	No
OpenAI	platform.openai.com/api-keys	gpt-4o-mini	No

Table 8.2: Supported providers

A different model can be chosen for each provider in the settings (Section 8.8). The provider or key can be changed at any time with **BluePrint: Change LLM Provider / API Key**.

8.4 Setting Up a Project

Open the project folder in VS Code (**File → Open Folder...**). If the folder is not yet a git repository, run `git init` in it first. Then start the setup wizard by clicking **BluePrint: Not initialized** in the status bar and

choosing **Initialize**, or by running **Blueprint: Initialize Project** from the command palette (Ctrl+Shift+P). The wizard has four steps (Figures 6.7 and 6.8):

1. **LLM provider.** Choose a provider, paste the API key and click **Next**. Blueprint sends one short test request; if the key is rejected, the model is unavailable or the provider's project is blocked, the wizard says so and the key is not saved.
2. **Team and roles.** This step appears only if the project has no roles file yet. Choose **Just me** (you become the only Architect), **A team** (list the other Architects and, optionally, the Developers, one e-mail per line) or **Skip for now** (everyone is an Architect). Roles are explained in Section 8.6.
3. **Describe your system.** Enter a project name and describe the system in plain language: what it does, its main parts, and the technologies already chosen. Click **Generate Architecture**. After 10 to 30 seconds Blueprint writes docs/ARCH.md with the components, data flow, constraints and open questions it found; every component starts as *Planned*.
4. **Surface constraints.** Click **Surface Constraints** to answer up to five questions about the architecture (for example data storage or external services), or **Done** to skip them. For each concrete answer Blueprint may ask one follow-up and then drafts a single ADR covering both answers. Edit the draft's title, context, decision and consequences as needed, then **Save ADR** or **Discard**. Tentative answers ("not sure yet") produce no ADR, and **Skip** moves to the next question.

When the wizard finishes, the Blueprint sidebar appears and the status bar shows **Blueprint**. The files in Table 8.3 are created. They belong to the project: commit them so that the whole team shares the same architecture and decisions.

File or folder	Contents
docs/ARCH.md	The architecture document, readable on its own and safe to edit by hand
docs/adr/0001-<title>.md, ...	One Markdown file per Architectural Decision Record
.blueprint/arch.json	The architecture in structured form, kept in step with ARCH.md
.blueprint/adr-index.json	Index of all ADRs with their status and cached embeddings
.blueprint/roles.json	The team's Architects and Developers (if roles were set up)
.blueprint/audit-log.json	The decision audit trail
.blueprint/history/	Earlier versions of ARCH.md, used by Revert

Table 8.3: Files created by Blueprint in the project

8.5 Using Blueprint

The **Hub** (Figure 6.6) is the starting point for every task. Open it by clicking Blueprint in the status bar, with the Blueprint button in the editor title bar, or with **Blueprint: Open Hub**. Every task is also available as a Blueprint: command in the command palette (Table 8.5).

8.5.1 Checking a prompt before coding

Use the Pre-Check before asking an AI assistant to write code, to catch a request that would break an architectural decision (Figure 6.12).

1. In the Hub, click **Pre-check a prompt** (or run **BluePrint: Pre-Check Prompt Against Architecture**).
2. Type or paste the prompt you intend to give the assistant and click **Check Architecture** (or press Ctrl+Enter).
3. If conflicts are found, the panel lists each concern with the ADR it relates to and suggests a revised prompt; click **Use this prompt** to adopt it, or **Revise prompt** to edit your own.
4. Choose the ADRs to include and click **Copy prompt with context**. Paste the copied text into the AI assistant, so that it sees the relevant decisions and constraints along with the request.

8.5.2 Reviewing code changes

A compliance review checks every uncommitted change in the project (staged, unstaged and new untracked files; files ignored by .gitignore are skipped) against ARCH.md and the accepted ADRs. Start a review in any of three ways:

- click **Review my changes** in the Hub, or run **BluePrint: Review This Change Architecturally**;
- click **Review** in the notification *"New file detected — review architectural impact?"* that appears when new files are created (this can be turned off, Section 8.8);
- click **BluePrint: Violation** in the status bar, which brings back the last review.

The Compliance Review panel (Figures 6.9 to 6.11) shows four kinds of result. Resolve each item with the buttons on its card:

Result	Meaning	Actions
Violation	The change breaks a constraint or an accepted ADR; shown with its severity and location	• Update Architecture: the change is intended; give the reason and BluePrint records an ADR that updates the architecture. • Modify Code: the change is a mistake; fix the code and click Re-run review. Nothing is recorded.
Planned component now in code	The change implements a component that ARCH.md lists as Planned	• Mark as Implemented: ARCH.md shows the component as Implemented with its files; no ADR is needed. • It's a New Component Instead, if BluePrint matched the wrong component.
New component	The change adds something that is neither planned nor implemented	• Register: add it to ARCH.md and record an ADR, optionally with extra context. • Dismiss, if it is not architecturally significant.
Checked file	An added file that was examined and found not to be a new component, with the reason	• Register as new component anyway, to overrule the verdict.

Table 8.4: Compliance review results and how to resolve them

When every violation is handled, the status bar returns to its normal state. If the panel reports *No changes found*, it explains why: the folder is not a git repository, the folder is ignored by an enclosing repository, or there really are no uncommitted changes.

8.5.3 Recording a decision

Decisions made outside a review, for example in a design discussion, are recorded with **Add a decision** in the Hub, the + button of the Architectural Decisions view, or **Blueprint: Add Decision (ADR)** (Figure 6.13). Describe the decision in plain words; Blueprint drafts an ADR to edit before clicking **Save ADR**. If the decision replaces an existing one, choose that ADR (Blueprint may suggest it): the old ADR is marked superseded and its constraint in ARCH.md is replaced. The wizard's constraint questions can be answered again at any time with **Blueprint: Answer Constraint Questions**.

8.5.4 Browsing decisions and the audit trail

The Blueprint sidebar (Figure 6.5) has two views. **Architectural Decisions** lists every ADR grouped by status, with decisions awaiting approval first; click an ADR to open it. The buttons at the top of the view add a decision, **search** for one ADR by text, **filter** the list, and clear the filter. **Audit Trail** lists the 20 most recent events; the full trail (Figure 6.15) has a Decisions tab and an Activity tab, with search and filters, and links to the ADRs and changed files of each event.

8.5.5 Viewing and maintaining ARCH.md

Click **View architecture** in the Hub to open the ARCH.md Viewer (Figure 6.14). It shows the rendered document with the sections changed by the five most recent changes highlighted, and why each change was made; the Components table shows which components are Planned and which are Implemented. **Edit source** opens the Markdown file; manual edits are kept when Blueprint later updates the document. Architects can also **Revert** ARCH.md to an earlier version (a diff is shown before confirming) and **Regenerate** it from the current codebase, which rewrites the components and data flow while keeping every constraint. ARCH.md is saved to the history before every change, so a regeneration can be undone with Revert.

8.6 Working in a Team

Blueprint has two roles, identified by each person's git config user.email. **Architects** approve decisions, and their own decisions take effect at once. **Developers** can use every feature, but the decisions they record wait in an approval queue and change nothing in ARCH.md until an Architect approves them. If no roles are set up, everyone is an Architect. The status bar tooltip and the title of the Architectural Decisions view show the current user's role.

1. An Architect sets up the roles in the wizard, or later with **Blueprint: Configure Team & Roles** (Figure 6.16), and commits `.blueprint/roles.json` together with `docs/`.
2. Team members pull the repository and install Blueprint with their own API key (**Blueprint: Change LLM Provider / API Key**). The project is already initialised, so they do not run the wizard.
3. When a Developer records a decision, it appears under *Pending approval* in the sidebar, the status bar shows the number of pending decisions, and the Hub shows a banner.
4. An Architect opens the pending ADR and clicks **Approve** or **Reject** (the check and cross buttons on the item, or the commands of the same name). A rejection needs a reason. On approval, ARCH.md is updated and the event is recorded in the audit trail.

Changes that teammates push (new ADRs, approvals, edited roles) appear in the sidebar and the open panels within about a second of a git pull, without reloading. Roles are an agreement rather than a lock: anyone who can push can edit `roles.json`, so protect it with code review, for example a CODEOWNERS rule.

8.7 Command Reference

Command	What it does	Who
Initialize Project	Runs the setup wizard (asks before overwriting an existing ARCH.md)	Architects, once roles exist
Open Hub	Opens the Hub	Everyone
Review This Change Architecturally	Runs a compliance review of the uncommitted changes	Everyone
Pre-Check Prompt Against Architecture	Checks a prompt for an AI assistant against the decisions	Everyone
View Architecture (ARCH.md)	Opens the ARCH.md Viewer	Everyone
Add Decision (ADR)	Drafts an ADR from a plain-language description	Everyone
Answer Constraint Questions	Reopens the wizard's constraint questions	Everyone
Open ADR Browser / Search ADRs / Filter ADR Browser / Clear ADR Filter	Shows, searches and filters the list of decisions	Everyone
View Decision Audit Trail	Opens the full audit trail	Everyone
Approve / Reject Pending Decision	Accepts or rejects a decision awaiting approval	Architects
Revert ARCH.md to Earlier Version	Restores an earlier ARCH.md after showing the difference	Architects
Regenerate ARCH.md from Codebase	Rewrites ARCH.md from the current code, keeping constraints	Architects
Configure Team & Roles	Shows the roles; Architects can edit them	Everyone (edit: Architects)
Change LLM Provider / API Key	Changes the provider and key on this machine	Everyone

Table 8.5: BluePrint commands (command palette)

8.8 Settings

Open **File** → **Preferences** → **Settings** and search for *BluePrint*.

Setting	Default	Effect
blueprint.autoReviewOnNewFile	On	Offer a review when new files are created in the workspace

blueprint.model.gemini, .groq, .openrouter, .anthropic, .openai	Empty (built-in default, Table 8.2)	The model to use with that provider; set it if the default model is retired or a different model is wanted
---	-------------------------------------	--

Table 8.6: BluePrint settings

8.9 Troubleshooting

Problem	Cause and solution
The review says <i>No changes found</i>	Read the reason shown. If the folder is not a git repository, run git init. If it is ignored by an enclosing repository, give the project its own repository. Otherwise there are no uncommitted changes to review.
The API key is rejected	Check that the key was copied completely and belongs to the chosen provider. If the message says the project has been denied access, the key is valid but the provider has blocked its project; create a key in a new project or use another provider.
<i>Model not found</i> (HTTP 404)	The provider has retired the default model. Set blueprint.model.<provider> to a current model name (Section 8.8).
<i>Rate limit</i> or time-out errors	BluePrint retries automatically; if it still fails, wait a minute and click Try again , or switch to another provider.
Approve, Reject, Revert or Regenerate are missing	These are Architect-only. Check the role in the status bar tooltip; an Architect can change it in Team & Roles.
The role shows <i>unknown</i>	Set the git e-mail address with git config user.email you@example.com.
ADR ranking seems less accurate offline	The embedding model could not be downloaded or loaded, so BluePrint uses word-based ranking. Connect to the internet once so the model can be downloaded, or install the package for your platform instead of the universal one.

Table 8.7: Common problems and solutions

8.10 Uninstalling

Open the Extensions view, find BluePrint, and choose **Uninstall**. The project's files (docs/ARCH.md, docs/adr/ and .blueprint/) are left in place; they are plain Markdown and JSON and remain readable without the extension. Delete them by hand to remove every trace of BluePrint from a project.

9. Conclusion

This report presented Blueprint, a Visual Studio Code extension that keeps a project's architecture and its architectural decisions next to the code and checks AI-assisted development against them. It addresses three problems that arise when AI assistants write much of a system's code: architectural drift, loss of design intent between sessions, and loss of developer control over architectural decisions. Blueprint answers them with a Constraints → Conformance → Knowledge cycle. Constraints are captured through a guided conversation that produces ARCH.md and Architectural Decision Records (ADRs). Conformance is checked before code is written, by comparing the developer's prompt with recorded decisions, and after it is written, by a two-pass review of uncommitted changes that reports violations and new, unplanned components. Knowledge grows with every resolution, since each one becomes an ADR or an update to the living architecture document, supported by version history, a team approval queue and an audit trail.

The system is built as a set of language-model agents coordinated by an orchestrator, with model output always validated and every decision left to a person. It supports five model providers, ranks past decisions with local embeddings that need no extra API, and runs inside the editor developers already use. All 169 automated tests pass, model-dependent behaviour was confirmed in repeated live runs, and the defects found in manual testing were fixed with regression tests. The project shows that architectural knowledge need not be a document written once and forgotten: it can be kept current and applied at the moment AI-generated code enters a project, while the developer stays in charge of every decision.

9.1 Limitations

Blueprint's main limitation is its dependence on the language model. Detecting violations and new components is a judgement the model makes, and its answers vary between runs and models. Strict output formats, validation and repeated live tests reduce this, and every verdict can be overruled, but the tool can still miss a problem or raise a false one. Live tests were run mainly on one model (openai/gpt-oss-120b on Groq), and the default models of the other providers have not been verified.

The tool also has practical constraints. Reviews require a git repository, since changes are found by comparing against the last commit. Every check needs an internet connection and an API key, and summaries and excerpts of changed code are sent to a third-party provider, which may be unacceptable for confidential code. Each pass sees only a summary and a diff excerpt of at most 6,000 characters, so very large changes are partly judged from the summary alone. Structural analysis with Tree-sitter covers eight languages, and others fall back to simpler pattern matching. Roles are read from a file in the repository, so their protection relies on the team's code review.

Finally, validation is incomplete. The manual acceptance checklist has been only partly run, the Linux and macOS packages have been built but not run, and regenerating ARCH.md from a codebase has received the least testing. Blueprint has not yet been evaluated with real development teams.

9.2 Future Work

The most important next step is an empirical evaluation with development teams on real projects, measuring how often Blueprint's findings are correct and useful and whether it reduces architectural drift. A second direction is closer integration with AI assistants: running the pre-check automatically on prompts inside the editor and giving assistants direct access to relevant ADRs as context, so the decision record shapes generation instead of only reviewing it. A third is continuous integration support, running the compliance review on pull requests so that changes made outside the editor are also checked and results are visible to reviewers. Finally, supporting locally hosted language models would remove the privacy concern by keeping code on the developer's machine.

References

- Jansen, A., & Bosch, J. (2005). Software architecture as a set of architectural design decisions. In *Proceedings of the 5th Working IEEE/IFIP Conference on Software Architecture (WICSA 2005)* (pp. 109–120). IEEE. <https://doi.org/10.1109/WICSA.2005.61>
- Kitayama, S. (2026). Rediscovering Architectural Decision Records: How Persistent Design Context Improves LLM Code Generation.
- Konrad, P. M., Adam, T. L., Terrenzi, R., & Ayvaz, S. (2026, June). Architecture Without Architects: How AI Coding Agents Shape Software Architecture. In *2026 IEEE 23rd International Conference on Software Architecture Companion (ICSA-C)* (pp. 463-468). IEEE.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474.
- Pressman, R. S., & Maxim, B. R. (2015). *Software engineering: A practitioner's approach* (8th ed.). McGraw-Hill Education.